

ARTDINK

BASICSTUDIO
「パワフルゲーム工房」

REFERENCE MANUAL

リファレンスマニュアル

Introduction

This reference manual explains the commands, functions, and related terminology that can be used in "EZ-BASIC."
*For explanation of other tools, consult the "User's Manual".

Table of Contents

• About this Book.....	2
• Basic commands.....	3
• Direct commands.....	25
• Extended commands.....	32
• 2D Graphics related.....	43
• 3D Graphics related.....	61
• Sound.....	82
• String functions.....	86
• Numeric functions.....	92
• Extended functions.....	101
• Index.....	106

Figures

• Fig. 1 Variables.....	3
• Fig. 2 Arrays.....	6
• Fig. 3 Multi-dimensional arrays.....	6
• Fig. 4 Screen composition.....	32
• Fig. 5 Screen coordinate system.....	43
• Fig. 6 Z-buffer method.....	44
• Fig. 7 Z-buffer issue.....	44
• Fig. 8 Z-sort.....	45
• Fig. 9 Palette and offset values	51
• Fig. 10 Texture coordinate system.....	54
• Fig. 11 Model display.....	61
• Fig. 12 Parent-child models.....	62
• Fig. 13 Fog settings.....	64
• Fig. 14 CAMROT.....	67
• Fig. 15 CAMVEC.....	68
• Fig. 16 FOGSET	69
• Fig. 17 LIGVEC	75
• Fig. 18 MODROT	80
• Fig. 19 Controller button numbers.....	104
• Table 1 JIS Codes.....	86
• Table 2 Key Codes.....	102
• Table 3 Controller button output	104

*This book will not be reissued, so please keep it safe so as not to lose it.

About this book

■ Composition

This manual is divided into the following sections according to the command functions.

- Basic commands: These are basic commands in BASIC, such as control structures and decision structures.
- Direct commands: Commands that have direct functions, such as editing programs and manipulating files.
- Extended commands: These are instructions that handle extended functions specific to EZ-BASIC, such as split screen display and controller vibration.
- 2D graphics related: commands that handle graphics such as backgrounds and primitives.
- 3D graphics related: commands that handle models, cameras, lights, etc.
- Sound: These are commands related to sounds such as background music and sound effects (SE: Sound Effects).
- String functions: Functions related to processing strings.
- Numeric functions: Mathematical functions and functions that process numbers.
- Extended functions: These functions handle extended functions specific to EZ-BASIC, such as collision detection for 3D objects and input from the keyboard.

■ How to read this book

■ GOTO		Continuing execution from the specified line
◆Format GOTO line number	"Format": Shows appropriate notation for the command. <i>Italics</i> indicates variables input by the user.	
◆Explanation • Continue execution from line number. The line number must be specified as an unsigned integer. • Specifying a line number that does not exist will result in an error. • You can also add a label as the line number.	"Explanation": Explains the function of the command.	
◆Related topics ON-GOTO (see p.19) LABEL (see p. 35)		Related topics: Listing of related functions/topics.
◆Example/ Using GOTO <pre>100 X=X+1 110 PRINT "*" 120 IF X<5 THEN GOTO 100 130 END</pre> If the value of variable X is less than 5, execution proceeds to line 100. <Execution results> * * * *		"Example": An example of usage.

*When entering and running the program shown in an example, execute the INIT command (see p.34) and the NEW command (see p.28) to reset the interface settings to their initial state.

■ Formatting rules

<i>Italics</i>	Indicates items for which the user must specify values or variables when used.
START+SELECT	This means pressing the START and SELECT buttons simultaneously.
{FIRST LAST}	Indicates a choice between FIRST and LAST.
[ABC]	Indicates that ABC can be omitted.
ABC...	Indicates that ABC is repeated 0 or more times.
(further lines)	Indicates the repetition of 0 or more program lines.

" " means to insert a half-width space.

Basic Commands

Program Description

◆Specifications

- A program consists of lines containing statements, each beginning with a line number.
- The range of integers that can be used as line numbers is from 1 to 999999999. 0 and negative numbers cannot be used.
- Programs are sorted in ascending order of line numbers and executed in ascending order of line numbers.
- Multiple lines cannot have the same line number. If a number is reused, the latest line will be assigned that line number.
- A program can be written up to 20,000 lines long.
- Each line can contain up to 132 characters, including the line number.
- Line numbers can be preceded by zeros. For example, line number 10 can be written as 00010.
- Variable names, command words, subprogram names, and other function words are not case sensitive.
- Not compatible with JIS standard FullBASIC or BASIC made by other companies.

◆Variables

- Variables have the ability to store numeric or string values.
- Variables that hold numbers are called numeric variables, and variables that hold strings are called string variables.

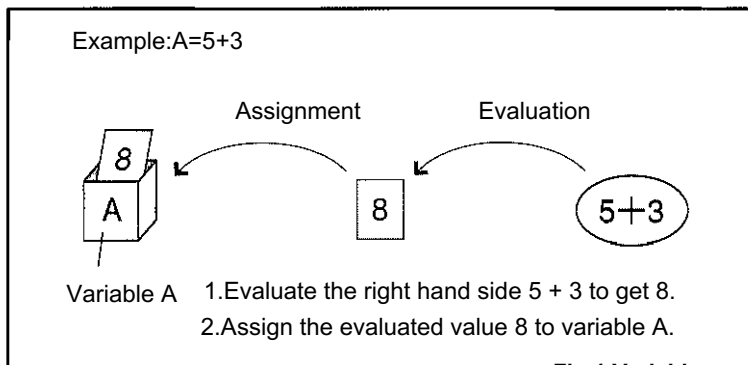


Fig.1 Variables

Putting a value into a variable is called "assignment."

Example

$A=5+3$

This expression evaluates the right-hand side ($5 + 3$) and assigns the value 8 to the variable A on the left-hand side.

- You can use up to 1000 numeric and string variables in total.
- Variable names are recognized up to 10 characters long, including the \$ sign for string variables. Any characters beyond that are ignored.
- Variable names can contain letters, numbers, and the underscore symbol (`_`). However, they cannot start with a number.
- Words reserved as function words, such as commands, cannot be used as variable names.

◆Full-width Kana and Kanji

- EZ-BASIC allows you to use full-width characters, such as Japanese, in your programs.
- Full-width characters are two half-width characters long. When you delete a full-width character, one half-width character is deleted and the remaining half-width character is replaced with a space.
- Can only be used as a comment or string value. Cannot be used as a variable name, subprogram name, etc.
- For information on entering full-width kana and kanji, please refer to the Kana-Kanji Conversion section of the User's Manual.

■ Operators

◆ Arithmetic operators

These are operators for performing numerical calculations. Operations follow standard algebra.

- +** Addition, positive sign
- Subtraction and negation, negative sign
- *** Multiplication
- /** Division. Dividing by 0 is an error.
- ^** Power. 0^0 becomes 1.
- e** Scientific notation. $3e2$ represents 3×10^2 , $5e-2$ represents 5×10^{-2} .

• Example

$A=23+5*12-36/3^2$ Arithmetic operator example. The value 79 is assigned to the variable A.

$A=-A$ Negation example. Variable A's sign is inverted.

$A=3e+2$ Scientific notation example. $3.0 * 10^2$ is assigned to A.

◆ Comparison operators

The left and right sides of the operator are compared, and the syntax is evaluated as true (the comparison operation matches) or false (the comparison operation does not match).

- =** The left and right values are equal.
- >** The left value is greater than the right value.
- <** The left value is less than the right value.
- >=, >=** The left value is greater than or equal to the right value.
- <=, <=** The left value is less than or equal to the right value.
- <>, <>** The left and right values are not equal.

◆ Logical operators

The left and right sides of the operator are compared to evaluate the truth value of the syntax.

- AND** The left side of the expression is true, and the right side of the expression is also true (logical AND).
- OR** The left side is evaluated as true or the right side is evaluated as true (logical OR).
- NOT** Negates the evaluation value of the right-hand side (logical negation).

• Example

IF $A < 8$ AND $C = D$ THEN PRINT "HAPPY"

IF NOT $A = B$ THEN PRINT "UNHAPPY"

IF $A = 8$ OR $C < D$ AND $C = 20$ THEN PRINT "HAPPY?"

IF $(A = B \text{ OR } C < D)$ AND $C = 10$ THEN PRINT "HAPPY!"

If $A < B$ and $C = D$, the THEN clause is executed.

If $A = B$ is not true, the THEN clause is executed.

If $A = B$ or $C < D$ and $C = 10$, the THEN clause is executed.

If $A = B$ or $C < D$ and $C = 10$, the THEN clause is executed.

◆ Order of operations and associativity

- Expressions are normally evaluated from left to right, with the priority of operators or parentheses determining the order of evaluation.
- By enclosing it in parentheses (), you can give priority to the calculation.
- Operator priority
The stronger the coupling, the higher the priority of evaluation.

Priority	Operator
Strong	()
	^
	+ - sign NOT
	*
	/
	+ - add/subtract
	< <= <> = > >=
Weak	AND OR

• Example

IF NOT $(A*3-4 \leq B)$ AND $((-A+B > C^3) \text{ OR } (B=C))$ THEN 250

This is evaluated in the following order:

First, the arithmetic expressions are evaluated according to their associativity, and then each comparison expression is evaluated.

(Formula 1) : $A*3-4 \leq B$ The value of $A*3-4$ is equal to or less than the value of 8.

(Formula 2) : $-A+B > C^3$ The value of $-A+B$ is greater than the value of C^3 .

(Formula 3) : $B=C$ The value of 8 and the value of C are equal.

Next, logical operations are performed on the results of each expression.

IF NOT (Formula 1) AND ((Formula 2) OR (Formula 3)) THEN 250

If (Formula 1) is not true and (Formula 2) is true or (Formula 3) is true, the THEN clause is executed.

■ Numeric Values

◆ Numeric representation

- You can use numbers (0 to 9), decimal points (.), signs (+-), and exponent symbols (e or E) to represent numerical values .
- Numbers are expressed in the following format:
 - (1) No decimal point, no exponent format. e.g. 500, -2300
 - (2) Format with decimal point and no exponent, e.g. 12.34, .45
 - (3) Format with decimal point and exponent, e.g. -2.9090 e+20
 - (4) No decimal point, but with exponent. 1 e+14, -5 e-30If the number of digits is large enough, it is expressed in format (3) or (4).
- If the integer part of a number expressed in decimal format is zero, the zero can be omitted. For example, -0.5 can be written as -.5.
- Numbers are stored in 32-bit floating-point binary format, with a precision of approximately 6 decimal digits.
When adding sufficiently large and small numbers, the smaller may be rounded to 0 due to the number of significant digits.
When dealing with decimals, errors may occur even within the number of significant digits because they are expressed in binary.
- The range of values that can be treated as numbers is -3.40282e + 38 to 3.40282e +38 .
The exponent range is approximately e-37 to e+38 .

◆ Numeric variables

- Only numeric values can be assigned. An error will occur if you try to assign a non-numeric value, such as a string.
- The values that can be held are -3.40282e +38 to 3.40282e+38 .

■ Strings

◆ Strings

- Strings can contain letters, numbers, symbols, and full-width kana and kanji.
- By writing double quotation marks (") before and after a set of characters, it will be considered a string.

◆ String variables

- Adding a \$ sign to the end of a variable name will cause it to be interpreted as a string variable.
- If you try to assign a value other than a string, such as a number, an error will occur.
- A string variable can contain up to 512 characters. However, full-width kana and kanji characters are evaluated as two characters.

◆ String concatenation

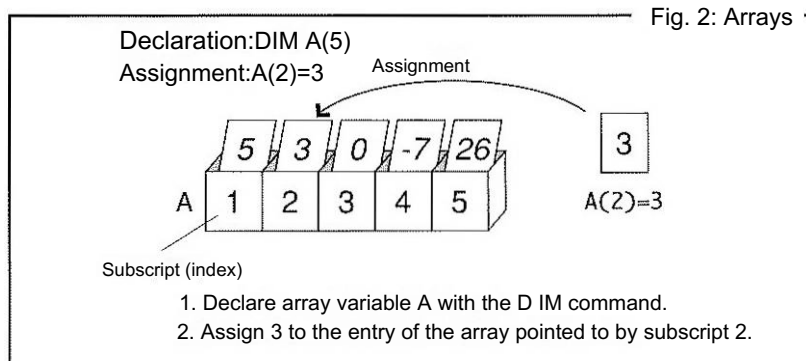
- You can obtain the value of concatenating strings using the string concatenation symbols (+ or &).
A\$="ABC"+" DEF"&"GHI" assigns the value "ABCDEFGHI" to the string variable A\$.
The + and & symbols used to concatenate strings have the same function.

◆ Substring specification

- You can use the substring specification to specify a part of a string stored in a variable.
- For the string variable A\$, you can use the format A\$(m:n) to indicate the string from the mth character to the nth character of the value stored in the string variable A\$.
If A\$="BASIC STUDIO", then A\$(8:11) represents the string "TUDI".
- You can assign a string to a specified range.
The string you assign will be replaced with the specified substring.
Therefore, if the string to be substituted is shorter than the specified substring, the entire string will be shortened, and if the string to be substituted is longer than the specified substring, the entire string will be lengthened.
When A\$="BASIC" is specified, executing A\$(1:4)="AB" replaces the value "BASI" pointed to by the substring specification A\$(1:4) with the string "AB", resulting in the string "ABC".
- With the substring specification A\$(m:n), when m>n, you can insert a string just before the mth character.
When A\$="SING", executing A\$(2:1)="TR" inserts the string "TR" immediately before the second character represented by the substring specification A\$(2:1), resulting in the string "STRING".

◆Using Arrays

- Arrays can store and handle a collection of numbers or strings.
Arrays are classified as either numeric arrays or string arrays.
- Arrays can be used by declaring them with the DIM command (see p.11).
- Each element stored in an array can be specified by a number called a subscript or index.
- If the array subscript contains decimal points, the decimal points are rounded off and the result is treated as an integer.



- Numeric arrays can only store numeric values in their elements. Attempting to assign any other value will result in an error.
- The maximum number of elements that can be stored in one numeric array is 1,000,000.
- String arrays can only store strings in the elements of the array. Attempting to assign any other value will result in an error.
- The maximum number of elements a string array can contain is determined by the amount of available memory.

◆Multidimensional arrays

- In addition to one-dimensional arrays, two-dimensional and three-dimensional arrays can also be declared.
- To declare a two- or three-dimensional array, write the array elements following the DIM command, separated by commas (,).

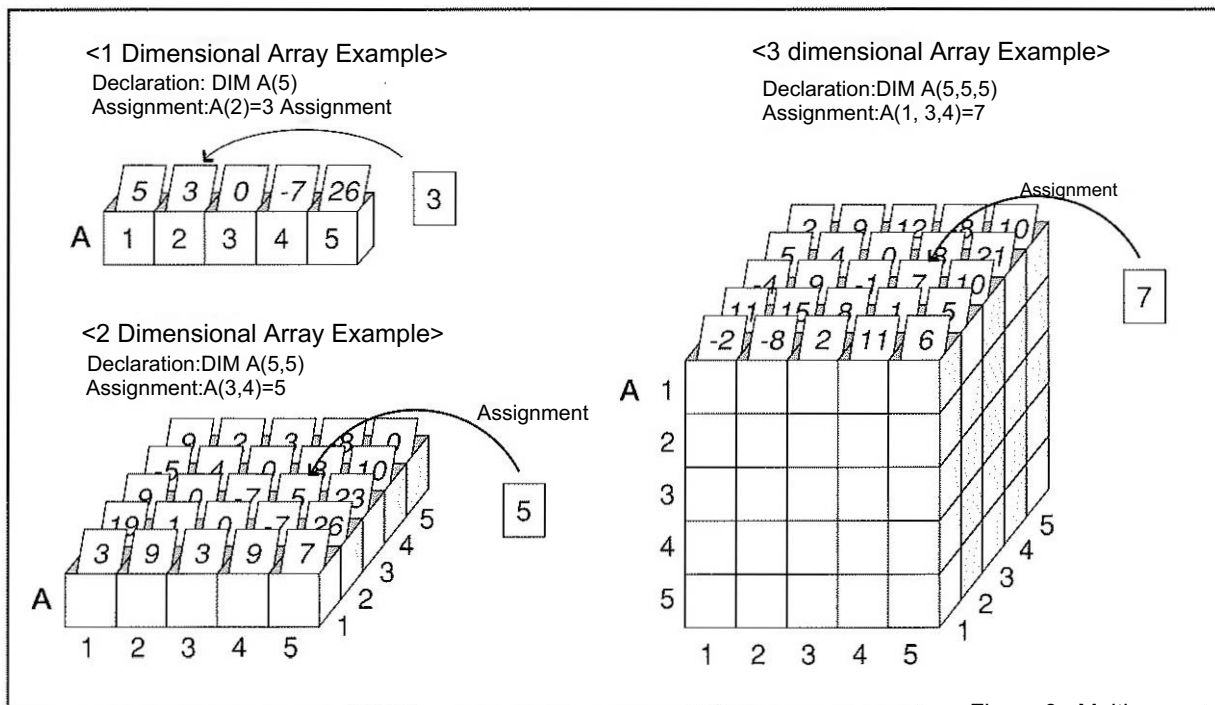


Figure 3 : Multi-dimensional array

◆ Subprogram Functionality

- A subprogram is a function that allows you to group together a set of commands with a certain function, and then execute the grouped commands by calling that subprogram.
- Subprograms can be freely defined by the user.
- A maximum of 1000 subprograms can be defined.

◆ Arguments

- Numbers, strings and variables can be passed as arguments to a subprogram.
By passing arguments to a subprogram, you can have it perform processing based on those arguments.

• Example

100 DO	
110 CALL CONTR(1)	The subprogram CONTR is executed, passing the number 1 as an argument.
120 CALL CONTR(2)	The subprogram CONTR is executed, passing the number 2 as an argument.
130 FSYNC 1	
140 LOOP	
150 SUB CONTR(P)	Subprogram CONTR definition part
160 TILEDISP P	Displays the tile primitive of the numerical value received in the argument P.
170 X=PAD(P,1)-PAD(P,2)	Checks the input of the controller connected to port P.
180 TILEPOS P,X+320,40,X+360,60,0	Updates the position of the tile primitives.
190 END SUB	End of subprogram definition.

...The subprogram CONTR is defined in lines 150 to 190.

...This subprogram manipulates 2 tile primitives (white squares on the screen) by pressing left and right on the controller.

...This subprogram uses one argument, the variable P. Based on the variable P, the subprogram determines the controller number and the tile primitive of which to control the position.

Depending on whether the value of P passed in lines 110 and 120 is 1 or 2, either the controller connected to port 1 or 2 will be used to direct the position of the tile.

- Arguments used temporarily for definition are called formal arguments. Arguments used at runtime are called actual arguments.
- The types (numbers, strings, etc.) and number of formal and actual arguments must be the same.

◆ Array arguments

- Arrays can be used as formal arguments in subprogram definitions. This is called a formal array declaration.
The format for a formal array declaration is the formal array variable name followed by parentheses. For a multidimensional array, write (number of dimensions - 1) commas (,) within the parentheses
- Format for dummy array declaration

SUB SP1(CH\$())	CH\$() is a temporary array declaration for a one-dimensional string array.
SUB SP2(AR(,,))	AR(,,) is a temporary array declaration for a three-dimensional numeric array.
- Format for passing arguments
When you run a program, you pass an array to a subprogram by writing only the array name.

CALL SP1(DM\$)	One-dimensional character string array DM\$ is the actual argument passed to SP1 above.
CALL SP2(DM)	The three-dimensional numeric array DM is the actual argument passed to SP2 above.

◆Values and References

- Arguments can be values such as numbers or strings, or variables.
- When an argument is passed a variable name, the argument becomes a reference to that variable.
By reference, any assignments to the formal argument are reflected in the variable you pass.

• Example

```
100 SUB SP1(A)           A subprogram that doubles the value of its argument
110  A=A*Z
120 END SUB
130 X,Y=5                Assign 5 to variables X and Y.
140 CALL SP1(X)          Passes variable X by reference.
150 PRINT X              The reference is used to assign the value to variable X, so the value becomes 10.
160 CALL SP1((Y))        Pass the value 5 for the variable Y.
170 PRINT Y              Y is not affected because only the value is passed.
180 END
```

<Execution result>

```
10
5
```

- If you want to pass the value of a variable instead of a reference, enclose the variable in parentheses ().

◆Recursion

- A subprogram can call itself, which is called a recursive call.

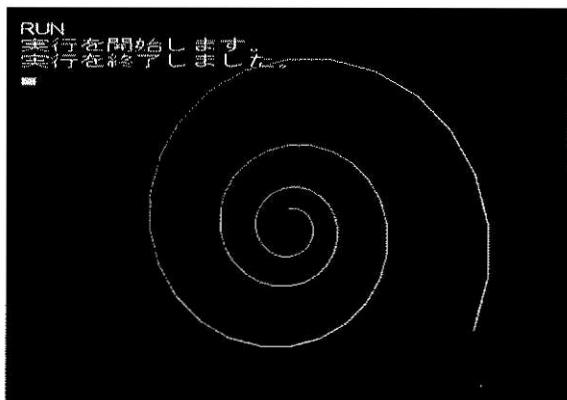
• Example

```
100 CALL RECURSION(1,320,224,0,6)
110 SUB RECURSION(ID,X,Y,R,L)
120  RX=X+L*COS(R)
130  RY=Y+L*SIN(R)
140  RR=R+RAD(15)
150  RL=L*1.03
160  LINEDISP ID
170  LINECOL ID,Y/2,X/3,ID*3,128
180  LINEPOS ID,X,Y/2,RX,RY/2,0
190 IF ID<80 THEN CALL RECURSION(ID+1,(RX),(RY),(RR),(RL))
200 END SUB
210 END
```

<Explanation>

100 Call the subprogram RECURSION.
110 The arguments are primitive number ID, line start point (X, Y), line end direction R, and line length L.
120 Calculates the line end point (RX, RY) from the arguments
140 Recursively calls RECURSION to calculate the angle of the next primitive.
150 Recursively call RECURSION to calculate the length of the next primitive.
190 If the primitive number is less than 80, it calls RECURSION itself with the arguments.
If it is equal to or greater than 80, the call is not made and the subprogram is terminated.

<Execution result>



◆Format

CALL SubprogramName [(Argument1,Argument2...)]

◆Explanation

- The contents of the subprogram defined with SUB-END SUB (see p.10) command are called and executed.
- You can give arguments to a subprogram and execute it. The arguments are written in parentheses, and if there are two or more arguments, they are listed separated by commas (.). (Example 2)
- If the number and type of arguments of the subprogram defined by the SUB-END SUB command do not match those of the subprogram called by the CALL command, an error will occur.
- When the execution of the called subprogram is completed, execution continues from the line following the CALL command.
- When an argument is passed as a variable name, the argument is passed to the subprogram by reference. (Example 3)
- A subprogram can call itself, i.e., it can make a recursive call.

◆Related topics

SUB-END SUB (see p.10)

•Example 1/ Example using a subprogram

```
100 CALL SUBP1           Call and execute subprogram SUBP1
110 END                 After executing the program at line 100, it ends at this line.
120 SUB SUBP1           Starting line of subprogram definition
130 PRINT "TEST"
140 END SUB             End line of subprogram definition.
```

•Example 2/ Subprogram with arguments

```
100 CALL SUBP2(3,5,9)   Execute the subprogram SUBP2 by passing 3, 5, and 9 as arguments.
110 CALL SUBP2(23,0,9)  Execute the subprogram SUBP2 by passing 23,0,9 as arguments.
120 END                 After executing line 110, execution ends with this line.
130 SUB SUBP2(A,B,C)    Start of subprogram definition
140 PRINT A+B+C         Print sum of A, B and C.
150 END SUB             End of subprogram definition
```

<Execution Result>

```
17
32
```

•Example 3/ Subprogram using references

```
100 A=5
110 B=5
120 CALL SUBP3(A,B)     Variable A is passed the subprogram by reference, and variable B is passed by value.
130 PRINT A,B           Shows how variables A and B have changed in the subprogram SUBP3.
140 END
150 SUB SUBP3(X,Y)      The starting line of the subprogram definition
160 X=10                Assign 10 to formal parameter X. At runtime, the value of variable A becomes 10 by reference.
170 Y=10                Assigns 10 to formal parameter Y. Variable B , which is passed by value, is not affected.
180 END SUB             End line of subprogram definition
```

<Execution Result>

```
10 5
```

◆Format

(1) Definition

```
SUB subprogram name [(Argument1, Argument2,...)]
    (series of lines)
END SUB
```

(2) Exit the subprogram

```
EXIT SUB
```

◆Explanation

- The series of lines from the SUB line to the END SUB line is defined as a subprogram (see p.7).
- A defined subprogram is called by writing the subprogram name followed by the CALL command (see p.9), and the defined sequence of lines is executed.
- You can use arguments in a subprogram by writing the formal arguments in parentheses after the subprogram name. If there are multiple formal arguments, they are listed separated by commas (,).
- Variables defined as arguments to a subprogram are treated as local variables that are valid only within that subprogram (Example 2). If the name of a variable defined outside the subprogram is the same as the name of a local variable, the local variable takes precedence. Local variables are destroyed when the subprogram finishes executing.
- When a subprogram is executed using the CALL instruction, if the actual argument is a variable, the dummy argument is treated as a reference to the actual argument. By reference, assignments to the dummy argument are reflected in the actual argument.
- When the EXIT SUB command is executed, execution of the subprogram ends and execution continues from the line following the line with the initiating CALL command.
An error occurs if the EXIT SUB command is written outside a subprogram.
- When program execution reaches a SUB line without being directed by a CALL, execution jumps to the line following END SUB.

◆Related topics

CALL (see p.9)

◆Example 1/ Example use of a subprogram

100 CALL SEISEKI("Hanano",85,70)	Pass the arguments "Hanano", 85, 70 and execute SEISEKI
110 CALL SEISEKI("Ichirota",23,40)	Pass the arguments "Ichirota", 23, 40 and execute SEISEKI
120 SUB SEISEKI(NAME\$,TEST1,TEST2)	Subprogram definition start
130 PRINT NAME\$;"'s grades"	
140 PRINT TEST1;"pts",TEST2;"pts"	
150 PRINT "Average", (TEST1+TEST2)/2;"pts"	
160 END SUB	Subprogram definition end
170 END	After line 110, the subprogram definition is skipped and this line is executed.

<Execution result>

```
Hanano's grades
85 pts 70 pts
Average 77.5 pts
Ichirota's grades
23 pts 40 pts
Average 31.5 pts
```

◆Example 2/ Local Variables

100 SUB SP1(A)	This variable A acts as a local variable.
110 A=A*2	This is an assignment to the local variable A.
120 END SUB	After the subprogram ends, the local variable A is destroyed.
130 A=4	This variable A acts as a global variable.
140 CALL SP1(5)	
150 PRINT A	Global variable A is displayed.

<Execution Result>

```
4
```

◆Format

- (1) Array declaration
DIM *arrayName* (*maximumValue*)
- (2) Array declaration with lower and upper bounds
DIM *arrayName* (*lowerBound* TO *upperBound*)
- (3) Multidimensional array declaration
DIM *arrayName* (*maximumValue*, *maximumValue*...)
- (4) Multiple array declarations
DIM *arrayName1*, *arrayName2*,...

◆Explanation

- Declares an array. By declaring an array, you can handle a collection of numbers or strings.
- Values stored in an array can be pointed to by numbers called subscripts (indexes).
- The *maximumValue* in parentheses after *arrayName* specifies the number of elements in the array to allocate. (Format (1))
- In parentheses following the array name, you can declare an array with index ranges from the *lowerBound* to the *upperBound*. The subscript (index) indicates the range from the lower bound to the upper bound. (Format (2))
- A multidimensional array (see p.6) can be declared by writing the maximum values in parentheses separated by commas (,) . Up to three-dimensional arrays can be declared. (Format 3)
A two- or three-dimensional array is declared by writing two or three maximum values.
- Multiple arrays can be declared by separating them with commas (,) (Format (4)).
- To declare a string array, add the "\$" symbol to the end of the variable name.
- If the limit is not specified, the implicit lower limit is 1. This can be changed to 0 using the OPTION BASE command (see page 19).

◆Related topics

OPTION BASE (see p. 19)

◆Example 1/ Array declaration example

DIM A(6)	Declare a one-dimensional numeric array with six values . The subscript range is 1 to 6.
DIM B(10,5)	Declare a two-dimensional numeric array.
DIM B\$(100)	Declare a one-dimensional string array.
DIM C(-5 TO 5)	Declare a one-dimensional numeric array. The subscript range is -5 to 5.
DIM D(10,10,10),D\$(50)	Declare a 3-dimensional numeric array of size 30x10x10 and a character array of size 50.

◆Example 2/ Using arrays

100 DIM TEST\$(0 TO 2)	Declare a one-dimensional string array TEST\$. Set the subscript range to 0 to 2.
110 TEST\$(0)="JAPANESE:75"	Assign the string " JAPANESE:75" to the element 0 of the array TEST\$.
120 TEST\$(1)="ENGLISH:50"	Assign the string " ENGLISH:50" to the element 1 of the array TEST\$.
130 TEST\$(2)="PHYSICS:50"	Assign the string " PHYSICS:50" to the element 2 of the array TEST\$.
140 PRINT TEST\$(0),TEST\$(1),TEST\$(2)	
150 END	

<Execution Result>

JAPANESE:75 ENGLISH:50 PHYSICS:50

◆Format

- (1) Repetition
 DO [{WHILE | UNTIL} *logical expression*]
 (series of lines)
 LOOP [{WHILE | UNTIL} *logical expression*]
- (2) Exiting repetition
 EXIT DO

◆Explanation

- Repeatedly executes the series of lines between DO and LOOP .
- You can write a logical expression that will be the condition for exiting the loop in the DO line or LOOP line.
- If you write a condition in the DO line, the condition to exit the loop is checked first.
 If you write a condition in the LOOP line, the loop will be executed at least once before the condition is evaluated to exit the loop.
- If you write a logical expression after WHILE, the loop will exit if the logical expression is false.
- If you write a logical expression after UNTIL, the loop will exit if the logical expression is true.
- When the loop is exited, execution continues from the next line after the LOOP statement.
- When EXIT DO is executed, it stops the execution and continues from the line after the LOOP statement (Example 2).

◆Related topics

FOR TO STEP~NEXT (see p. 14)

•Example 1/ Repetition Using DO-LOOP

```
100 DO                               Start of looped code
110  A=A+1
120  PRINT "HELLO"
130  LOOP UNTIL A>3                 Loop until variable A is larger than 3
140 END
```

•Example 2/ Using EXIT DO to leave a loop

```
100 DO
110  A=A+10                          Variable A is increased by 10 with each iteration.
120  PRINT A
130  IF A>50 THEN EXIT DO           If A exceeds 50 then exit the loop.
140 LOOP
150 END
```

<Execution Result>

```
10
20
30
40
50
60
```

•Example 3/ Repetition using WHILE

```
100 DO WHILE A<10 AND B<5           Repeat as long as the value of variable A is less than 10 and the value
110  A=A+2                          of variable B is less than 5 .
120  B=B+1
130  PRINT A,B
140 LOOP
150 END
```

<Execution Result>

```
2  1
4  2
6  3
8  4
10 5
```

◆Format

END

◆Explanation

- Ends program execution.

◆Format

(1) Repetition

```
FOR iterationVariableName = initialValue TO finalValue [STEP increment]
    (series of lines)
NEXT iterationVariableName
```

(2) Exiting repetition

```
EXIT FOR
```

◆Explanation

- The FOR block, which contains a series of lines from the FOR statement to the NEXT statement, is executed repeatedly.
- The iterations start from the *initial value* and continue until the value of the *iteration variable* exceeds the end value.
- When the NEXT statement is reached, the *iteration variable* is incremented and the program returns to the FOR statement. If the value of *iterationVariable* exceeds the end value, the program exits the FOR block and continues execution from the line following the NEXT statement. If the end value has not been exceeded, the sequence of lines in the FOR section continues to be repeated. The *iterationVariableName* following NEXT must be the same as the *iterationVariableName* used in the FOR statement.
- By writing an increment value after STEP, you can set the value to be added to the control variable in the NEXT statement (Example 2). If the STEP clause is omitted, the increment is implicitly set to 1.
- When an EXIT FOR statement is reached, the loop is exited. The EXIT FOR statement must be inside the FOR block. (Format (2))
- By writing a new FOR block within a FOR block, you can create a nested structure in which the inner FOR block is repeatedly executed by the outer FOR block (Example 3).
The control variable name used in the inner FOR block cannot be the same as the control variable name in the outer FOR block.
- If the initial value assigned to the control variable exceeds the limit, the lines in the FOR block will not be executed, and execution will begin with the line following the NEXT line.

◆Related topics

DO~LOOP (see p.12)

• Example 1/ Repeat statement using FOR TO ~ NEXT

```
100 SUM=0
110 FOR A=1 TO 100
120  SUM=SUM+A
130  IF A=10 THEN EXIT FOR
140 NEXT A
```

Iteration variable A starts as 1 and continues until it exceeds 100.
The value of iteration variable A is added to variable SUM with each loop.
The loop ends when the value of iteration variable A reaches 10.

• Example 2/ Example using step increment setting

```
100 SUM=0
110 FOR A=3 TO 1 STEP -1
120  SUM=SUM+A
130  PRINT "A:";A,"SUM:";SUM
140 NEXT A
150 PRINT "SUM=";SUM
160 END
```

Set the increment to -1.
The value of iteration variable A is added to variable SUM.

<Execution result>

```
A:3 SUM:3
A:2 SUM:5
A:1 SUM:6
SUM=6
```

•**Example 3/** A program that displays a multiplication table using nested FOR TO ~ NEXT statements

```
100 FOR A=1 TO 9
110 FOR B=1 TO 9
120   X=A*B
130   S$=STR$(X)
140   RES$=RES$+S$+" "
150 NEXT B
160 PRINT RES$
170 RES$=""
180 NEXT A
190 END
```

The iteration variable A is used to set the multiplication table row.
Iteration variable B is used for the multiplication table column.
Assign the answer to the multiplication table to the variable X.
Converts the value of the numeric variable X to a string and assigns it to S\$.
S\$ containing the answer is added to the string variable RES\$ which stores the multiplication table for each row , and the loop continues with the next number.
Displays the string of the completed column.
Empties the stored contents and continues the repetition to the next row.
Continue repeating to the next row.

<Execution Results>

```
1 2 3 4 5 6 7 8 9
2 4 6 8 10 12 14 16 18
3 6 9 12 15 18 21 24 27
4 8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81
```

GOTO

Continuing execution from a specified line

◆Format

GOTO *LineNumber*

◆Explanation

- Continue execution from the line *LineNumber*.
The line number you specify must be an unsigned integer.
 - Specifying a line number that does not exist in the program will result in an error.
- You can also use a label as the line number .

◆Related topics

ON - GOTO (see p.19) LABEL (see p.35)

• Example/ Using GOTO

```
100 X=X+1
110 PRINT "*"
120 IF X<5 THEN GOTO 100
130 END
```

If the value of variable X is less than 5, execution proceeds to line 100.

<Execution Results>

```
*
*
*
*
*
```


◆Format

- (1) Conditional branching of simple executable statements
 IF *logical expression* THEN { *instruction statement* | *line number* } [ELSE { *instruction statement* | *line number* }]
- (2) Conditional branching of multiple executable statements
 IF *logical expression* THEN
 [(series of lines)]
 [ELSE
 (series of lines)]
 END IF
- (3) Conditional branch with ELSEIF clause
 IF *logical expression 1* THEN
 [(series of lines)]
 ELSEIF *logical expression 2* THEN
 [(series of lines)]
 [ELSE
 (series of lines)]
 END IF

◆Explanation

- Selects the statement to be executed based on the evaluation result of a logical expression.
 One or more logical expressions can be listed.
- If the *logical expression* is true (the conditions are met), the statement following THEN is executed. (Example 1)
- If the *logical expression* is false (does not meet the conditions), if there is an ELSE , the statements following it are executed.
 If there is no ELSE , execution continues from the line following the IF statement.
- Multiple executable statements cannot be written immediately after a THEN or ELSE clause. To execute multiple lines, use format (2).
- If *logical expression 1* of the IF statement is false, execution moves to the ELSEIF line. (Example 3)
 If *logical expression 2* following ELSEIF is true, the sequence of lines following THEN on the ELSEIF line is executed.
 If *logical expression 2* is false, ELSE and the subsequent lines, if any, are executed.
 If there is no ELSE or the series of lines following it, execution continues with the line following the END IF line.

◆Related topics

SELECT CASE (see p.23)

• Example 1

```
100 SCORE=50
110 IF SCORE >=70 THEN PRINT "Pass" ELSE 130
120 END
130 PRINT "Fail"
140 END
```

<Execution result>

Fail

<Explanation>

- 110 If the variable SCORE is equal to or greater than 70, it displays "Pass"; if not, it moves to line 130.
 130 If the ELSE clause is executed on line 110 , execution will move to this line and display "Fail".

• Example 2

```
100 FOOD=1
110 IF FOOD=1 THEN
120 CAL=500
130 PRINT "Miso Saba"
140 ELSE
150 CAL=650
160 PRINT "Omelette"
170 END IF
180 END
```

If FOOD is 1, the THEN section in lines 120 to 130 is executed.

If FOOD is not equal to 1, the ELSE section on lines 150 to 160 is executed.

<Execution result>

Miso Saba

• Example 3

```
100 MATH=40
110 ENGLISH=60
120 IF MATH>=70 AND ENGLISH>=75 THEN
130 PRINT"Pass"
140 ELSEIF MATH<50 OR ENGLISH<50 THEN
150 PRINT"Retake"
160 ELSE
170 PRINT"Successful"
180 END IF
190 END
```

<Execution result>

Retake

<Explanation>

120 If variable MATH is greater than or equal to 70 and variable ENGLISH is greater than or equal to 75, line 130 is executed.
140 If the logical expression in line 120 is false and MATH is less than 50 and ENGLISH is less than 50, line 150 is executed.
160 If the logical expression in line 140 is false, line 170 is executed.

INPUT

Getting input characters from the keyboard

◆Format

INPUT [PROMPT *string* :] *variableName* [, *variableName* 2, *variableName* 3...]

◆Explanation

- Assign data entered from the keyboard to a variable.
- If *variableName* is a numeric variable, only numeric values can be assigned.
Characters can be assigned to string variables. In this case, numbers are also treated as strings.
- Input is completed by pressing the ENTER key.
- Writing a *string* after PROMPT will display the string when input occurs. The *variable name* to assign is written after a colon (:).
- You can assign keyboard input data to multiple variables by separating the variable names with a comma (,).
When inputting the information for multiple variables, separate each with a comma (,).

*To input data using this command, a USB keyboard must be connected to the USB terminal.

◆Related topics

INKEY\$ (see p.101)

PAD (see p.104)

• Example/ Example using keyboard input

```
100 INPUT PROMPT"Please type your name",NAME$,TEST1,test2,test3
110 PRINT NAME$;"'s average score:";(TEST1+TEST2+TEST3)/3
120 END
```

<Execution result>

```
Please type your name
Marin,45,30,77 (ENTER)
Marin's average score:50.6667
```

<Explanation>

100 Accepts input from the keyboard and assigns it to the variables NAME\$, TEST1, TEST2, and TEST3 .
110 The average of the variables TEST1, TEST2, and TEST3 is calculated and displayed.

◆Format

LET *variable name* [, *variable name 2*, *variable name 3*...] = *expression*

◆Explanation

- Calculates an expression and assigns the result to a variable. Assignments are made in the following order:
 1. Array subscripts (see p.6) and substrings (see p.5) are evaluated in order from left to right in the list of variable names.
 2. Evaluation of the right-hand side expression
 3. Assign the result of the expression to the list of variable names from the left.
- The LET command can be written without the word LET. If the word LET is omitted, the line will be treated as a LET command line.

•Example 1/ Using LET

```
LET P=3.14159
LET A,B,C=150
LET A(X,Y)=SIN(A)*N
LET A$="GREEN"
LET A$(2:4)="ABCDEF"
LET A$=B$ & "PLUS"
LET A$(2,3)(5:7)=$B
```

*The word LET can be omitted in all of the above.

Assign the value 3.14159 to the numeric variable P.
 Assign the value 150 to each of the numeric variables A, B , and C at the same time.
 Assign the result of the formula to two-dimensional numeric array A.
 Assign the string "GREEN" to the string variable A\$.
 Replace the substring from the second to fourth characters of A\$ with the string "ABCDEF".
 Assign a string that combines the value of B\$ and the string "PLUS" to string variable A\$.
 Replace the value of the fifth to seventh characters of element (2,3) of two-dimensional string array A\$ with the value of string variable B\$.

•Example 2/ Example showing the order of evaluation of LET

```
100 LET A$="ABCDE"
110 LET A$(1:2),A$(2:4)= "XYZ"
120 PRINT A$
130 END
```

After executing A\$(1:2)="XYZ" , execute A\$(2:4)="XYZ" .

<Execution Result>

XXYZDE

■ON~GOTO

Execution from line number using index

◆Format

ON *index* GOTO *line number* [, *line number*...] [ELSE { *instruction statement* | *line number* }]

◆Explanation

- Based on the *index* value, execution continues from the line with the line number following GOTO.
- The line numbers following GOTO are assigned in order from the left, starting with 1, 2, 3, etc. From among these, a line number in the same order as the number indicated by the index is selected, and execution continues from that line.
- If ELSE is followed by a statement or a line number and the index is less than or equal to 0 or greater than the number of line numbers in the list, execution continues from the statement or line number following ELSE.
- If there are no statements after ELSE, execution continues from the next line.

◆Related topics

GOTO (see p. 5)

LABEL (see p. 35)

• Example/ Using ON-GOTO

```
100 A=2
110 ON A GOTO 120,130,140 ELSE 150    The value of variable A, 2, causes execution to move to line 130.
120 PRINT "HOP"
130 PRINT "STEP"
140 PRINT "JUMP!"
150 END
```

<Execution Result>

```
STEP
JUMP!
```

■OPTION BASE

Minimum index setting

◆Format

OPTION BASE { 0 | 1 }

◆Explanation

- Sets the implicit lower bound of an array to 0 or 1.
- If you do not specify a lower bound for an array subscript, it is implicitly set to 1.
- The lower limit value setting takes effect from the array declaration using DIM after it is executed. (not retroactive)

◆Related topics

DIM (see p.11)

•Example 1

```
DIM A(10)
```

The indices of array A are 1 to 10. The size is 10. The lower limit, 1, is implicitly set.

•Example 2

```
100 OPTION BASE 0
110 DIM A(10)
120 END
```

Set the Lower Limit value to 0 .
The indices of array A are 0 to 10. The size is 11.

◆Format

PRINT *expression* [{ , | ; } *expression*...]

◆Explanation

- *expression* is evaluated and its value is displayed as text on the screen.
- You can write multiple expressions on the same line by using a comma (,) or semicolon (;) between them. A comma is replaced by two spaces when displayed. A semicolon is displayed as nothing.
- When a number is displayed, a space will automatically appear after the number.

• Example

```
100 PRINT 128*64
110 X=80
120 PRINT X
130 PRINT 12;34;"56"
140 PRINT
150 END
```

Evaluates the expression 128*64 and displays the result, 8192.

Displays the value of variable X.

Displays the numbers 12, 34 and the string "56" .

Nothing is displayed, only a line break is displayed.

<Execution result>

```
8192
80
12 34 56
...(Nothing is displayed, only a line break is displayed.)
```

◆Format

RANDOMIZE [*Seed*]

Item	Notes
Seed	Range -2e32 +2e32

◆Explanation

- Initializes the random number sequence used by the RND instruction (see p . 98). If you generate random numbers without executing RANDOMIZE, the values generated will always follow the same pattern each time the program is run.
- If you specify a random number seed, a random number sequence will be generated based on it.
The random number seed uses a number.
The random number sequence generated from a specified random number seed always follows a certain pattern.
- If you run the program without specifying a random number seed, the generated random number sequence will be undefined.

◆Related topics

RND (see p.98)

• Example

```
100 PRTNT RND
110 RANDOMIZE
120 PRINT RND
130 RANDOMIZE 23
140 PRINT RND
150 END
```

Generating random numbers without running RANDOMIZE will follow a fixed pattern.

If you do not specify a seed, an arbitrary sequence of random numbers will be generated.

If 23 is passed as the seed, a random number sequence will be generated based on that.

Generates values in a consistent pattern generated by the random number seed.

<Execution result>

```
RUN (enter)
0.51387
0.774261
0.818885
```

```
RUN (enter)
0.51387
0.288125
0.818885
```

◆Format

- (1) Loading data
`READ variable name [, variable name 2, variable name 3...]`
- (2) Data setup
`DATA data [data 2, data 3...]`

•Explanation

- The data list written in the DATA statement is assigned to a list of variables using the READ command.
 The data can be numeric or string.
- When a READ statement is executed, data is assigned to the variables in the variable name list of the READ statement, starting from the top of the data list in the DATA statement.
- If the data list in the DATA statement is insufficient compared to the list of variable names in the READ statement, an error occurs. Data in excess of the variables in the READ statement are ignored.
- A DATA statement can span multiple lines. If there are multiple DATA statements, the data list will be treated as one for the entire program.
- On a DATA line, the program does nothing and moves on to the next line.

◆Related topics

RESTORE (see p.22)

• Example

```
100 DATA 1,2,3,4,5
110 READ A,B,C
120 READ D,E,F,G
130 DATA 6,7,8
140 PRINT A,B,C,D,E,F,G
150 END
```

The DATA statement itself is not executed directly, execution moves to the next line.
 The first three items in the data list, 1, 2, and 3, are assigned to A, B, and C, respectively.
 READ continues, the four data 4, 5, 6, and 7 are assigned to variables D, E, F, and G.
 The unread data 8 will be ignored.

<Execution result>

```
1 2 3 4 5 6 7
```

■RESTORE

Change data read position

◆Format

RESTORE [*line number*]

◆Explanation

- Returns the data read position to the beginning of the data sequence that begins with the DATA command.
- If *line number* is specified, the data will be read from the beginning of the specified line.

◆Related topics

READ (see p.21) LABEL (see p.35)

•Example

```
100 READ A,B
110 PRINT "1:" ,A,B
120 READ B,C
130 PRINT "2:" ,A,B,C
140 RESTORE
150 READ A,B,C,D
160 PRINT "3:" A,B,C,D
170 RESTORE 200
180 READ A,B,C
190 PRINT "4:" A,B,C
200 DATA 100,110,120
210 DATA 130,140,150
220 END
```

Read data from the beginning of the data list, lines 200-210, into variables A and B.

Data is read from the current data read position into variables A, B, and C in order.

Returns the current data reading position to the beginning.

Data is read from the data read position at the beginning of the data sequence.

Set the data reading position to the beginning of line 200.

<Execution result>

```
1: 100 110
2: 120 130 140
3: 100 110 120 130
4: 100 110 120
```

■REM

Comments

◆Format

- (1) REM *Comment*
- (2) ! *Comment*

◆Explanation

- Write annotations for the program.
- When program execution reaches a comment statement, it proceeds to the next line without doing anything.
- A REM line does nothing and moves execution to the next line. (Format (1))
- The comment symbol "!" on the end of the line is interpreted as an end-of-line comment, and is not executed. (Format (2))

•Example

```
100 REM KEYCODE Test
110 DO ! Main loop
120 PRINT KEYCODE1
130 ! PRINT KEYCODE2
140 LOOP
150 END
```

Comments are used to describe the functionality of the program.
End-of-line comments are used to annotate lines.

By making the statement an end-of-line comment, execution is ignored.

◆Format

```
SELECT CASE Expression
CASE matching item [, matching item...]
    (series of lines)
[CASE matching item [, matching item...]
    (series of lines)]
...
[CASE ELSE
    (series of lines)]
END SELECT
```

◆Explanation

- *Expression* is evaluated, and if it matches the *matching item* following CASE, the series of lines in the CASE section are executed.
- You can write any of the following syntax in the *matching item* field:

(a) *Constant*

(b) *Constant TO Constant*

(c) *IS comparisonOperator Constant*

Writing variables or expressions other than (a) to (c) will result in an error.

- Matching items can be written consecutively, separated by commas (,).

In this case, if any of the *matching items* matches, the series of lines following CASE will be executed.

- When the execution of the set of lines in the CASE section is completed, or if none of the *matching items* match and there is no CASE ELSE line, execution continues from the line following the END SELECT line.

- By writing a CASE ELSE line and a series of lines following it, you can write what will happen if none of the *matching items* match.

◆Related topics

IF THEN~END IF (see p.16)

• Example

```
100 INPUT A
110 SELECT CASE A
120 CASE 77
130 PRINT "LUCKY!"
140 CASE 40 TO 70, 99
150 PRINT "CHANCE"
160 CASE IS < 0
170 PRINT "TRY AGAIN"
180 CASE ELSE
190 PRINT "SPECIAL CHANCE"
200 END SELECT
210 END
```

Select the line to execute depending on the value of variable A.

If the value is 77

If the value is between 40 and 70, or is 99

If the value is less than 0

If none of the above CASE statements apply

<Execution result>

?

-56(ENTER)

TRY AGAIN

You can only enter text if a USB keyboard is connected.

◆Format

STOP

◆Explanation

- Aborts program execution.
- The program can be resumed using the CONTINUE command (see p.25).
- Program execution can also be interrupted by pressing the Break key on the keyboard or by pressing the START and SELECT buttons.

◆Related topics

CONTINUE (see p.25)

• Example

```
100 DO
110 CODE=KEYCODE1
120 PRINT CODE
130 IF CODE=41 THEN STOP
140 LOOP
150 END
```

Assign the code of the pressed key to the variable CODE .

If the value of CODE is 41 (ESC key pressed), the STOP command is executed.

<Execution result>

```
...
0
0
41
```

実行を中断しました。 (The execution was interrupted)

Direct Commands

■CONTINUE

Program resumption

◆Format

CONTINUE

◆Explanation

- Resumes a program that has been suspended.
- When execution is interrupted by STOP (see p.24) or the Break key, execution can be resumed by the CONTINUE command.
- If you edit the currently suspended program, you will not be able to execute the CONTINUE command.

◆Related topics

STOP (see p.24)

■DELETE

Deleting a program line

◆Format

DELETE *line range*

Item	Notes
<i>line range</i>	Format:(a) <i>Line number</i> (b) <i>StartLine TO EndLine</i>
<i>StartLine</i>	Format:{ <i>Line number</i> FIRST LAST}
<i>EndLine</i>	Format:{ <i>Line number</i> FIRST LAST}

◆Explanation

- Deletes the range of rows specified by the line range .
- There are several ways to specify a line range :
 - (a) Line number
If you specify a *line number*, that line will be deleted.
 - (b) Start line TO End line
Deletes lines from *StartLine* to *EndLine*.
FIRST means the first line of the program.
LAST means the last line of the program.
- In the case of using a line range, the end line must be the same as or later than the start line.

• Example

```
DELETE 120
DELETE 240 TO 320
DELETE FIRST TO 500
DELETE 200 TO LAST
DELETE FIRST TO LAST
```

Delete line 120.
Delete lines 240 to 320.
Delete the from the start of the program to line 500.
Delete from line 200 to the last line of the program.
This will delete the entire program.

■FILES

Display files within a directory

◆Format

FILES [*DirectoryName*]

◆Explanation

- List the files in the directory where the data is stored.
- Displays a list of files in the specified directory.
- If you omit the directory name, a list of directories will be displayed.

◆Related Topics

KILL (see p.26)

SAVE (see p.30)

LOAD (see p.28)

DEVICE (see p.33)

◆Example

FILES

Displays a directory listing.

FILES "USER"

Displays a list of files in the "USER" directory.

■FKEY

Register a string to a function key

◆Format

FKEY *String FunctionKeyNumber*

Item	Note
<i>String</i>	Range:0-30 Characters
<i>FunctionKeyNumber</i>	Range:1~12

◆Explanation

- Set the string to be displayed when you press the function key.
- When you press the function key with the number corresponding to *functionKeyNumber*, *string* will be displayed.

◆Example

FKEY "RENUMBER AT 1000",6

Pressing the F6 key will display "RENUMBER AT 1000" on the text screen.

■KILL

Delete files

◆Format

KILL [*DirectoryName* /] *FileName*

◆Explanation

- Delete the file specified by *FileName*.
- To delete a file in a directory, write the directory name and a slash (/) followed by the file name.
If you omit *DirectoryName*, the "SYSTEM" directory will be used.

◆Related Topics

FILES (see p.26)

LOAD (see p.28)

SAVE (see p.30)

◆Format

LIST [*LineRange*] [REF *Keyword*]

Item	notes
<i>LineRange</i>	Format: (a) <i>LineNumber</i> (b) <i>StartLine</i> TO <i>EndLine</i>
<i>StartLine</i>	format: { <i>LineNumber</i> FIRST LAST }
<i>EndLine</i>	format: { <i>LineNumber</i> FIRST LAST }

◆Explanation

- Display the program on the screen.
- You can specify the range of lines of the program to display.
If not specified, all program lines will be displayed.
- *LineRange* can be specified in the following ways:
 - (a) Line number
If you specify *LineNumber*, that line will be displayed.
 - (b) *StartLine* TO *EndLine*
Displays lines from *StartLine* to *EndLine*.
FIRST means the first line of the program.
LAST means the last line of the program.
- If you write a *Keyword* after REF, it will search for and display lines that contain the keyword.
Keywords are case sensitive.
If you also specify *LineRange*, the lines that contain the keyword within that line range will be displayed.

◆Related topics

VIEW (see p.31)

•Example

LIST	Display the entire program
LIST FIRST TO 400	Display up to line 400.
LIST REF "REM"	Display all lines that include the string REM.
LIST 200 TO 400 REF "<>"	Display all lines in the range 200 to 400 that contain "<>"

■LOAD

Load a program

◆Format

LOAD [*DirectoryName* [/] *FileName* [MERGE]

◆Explanation

- Loads the program file specified by *FileName* from the specified device.
- The currently edited program will be discarded.
- The DEVICE command (see page 33) allows you to set the device from which the file will be read.
- If you specify *DirectoryName*, the program file specified in the file name will be loaded from that directory.
If the directory name is omitted, the file will be read from the "SYSTEM" directory.
- Specifying MERGE will merge the currently edited program with the loaded program.
Lines with the same line number will be overwritten by the loaded program.

◆Related topics

DEVICE (see p.33) SAVE (see p.30)

FILES (see p.26) KILL (see p.26)

◆Example

LOAD "TEST"

Loads the program file "TEST" from the "SYSTEM" directory of the specified device .

LOAD "USER/GAME1"

Loads the program file "GAME1 " from the "USER" directory of the specified device.

■MEMORY

Display free main memory

◆Format

MEMORY

◆Explanation

- Displays the remaining amount of main memory.

■NEW

Create a new program

◆Format

NEW

◆Explanation

- Discard the current program and edit a new one.
- Variables and arrays that were being used will also be destroyed.

◆Related Topics

INIT (see p.34)

■QUIT

Close "EZ-BASIC"

◆Format

QUIT

◆Explanation

- Exit "EZ-BASIC".
- The program currently being edited will be discarded.

◆Format

RENUMBER [*LineRange*] [AT *NewStartLine*] [STEP *Interval*]

Item	Note
<i>LineRange</i>	Format: (a) <i>LineNumber</i> (b) <i>StartLine TO EndLine</i>
<i>StartLine</i> format: { <i>LineNumber</i> FIRST LAST }	
<i>EndLine</i> format: { <i>LineNumber</i> FIRST LAST }	

◆Explanation

- Assign a new line number to a program line number.
- By specifying a line range, you can specify the range of line numbers of the program to be reassigned.
If no line range is specified, the entire program will be subject to the RENUMBER command.
- *LineRange* can be specified in the following ways:
 - (a) Line number
If you specify *LineNumber*, that line will be displayed.
 - (b) *StartLine TO EndLine*
Displays lines from *StartLine* to *EndLine*.
FIRST means the first line of the program.
LAST means the last line of the program.
- Specify the new starting line number by specifying AT followed by the new starting number.
If you do not specify *NewStartLine*, 100 is assigned.
- Specify the increment by which the newly assigned line numbers will increase by specifying *Interval* after STEP.
If you do not specify a step size, it will be assigned 10.
- If the newly assigned line number overlaps or straddles an existing line number, execution will not be possible.

◆Related Topics

LIST (see p.27)

VIEW (see p.31)

•Example

RENUMBER	Renumbers all lines in the program, starting with line number 100 and incrementing by 10.
RENUMBER 300 TO LAST	Renumbers line 300 to the end of the program, starting with line number 100, step 10.
RENUMBER AT 1000 STEP 25	Renumbers all lines in the program, starting with line number 1000 and incrementing by 25.

◆Format

RUN

◆Explanation

- Runs the currently loaded program.
- Program execution can be interrupted by using the STOP command (see p.24), the Break key on the USB keyboard, or the START + SELECT buttons.

◆Related topics

STOP (see p.24) CONTINUE (see p.25) END (see p.13)

◆Format

SAVE [*DirectoryName* /] *FileName*

◆Explanation

- Save the program you are currently editing.
- In *FileName*, specify the name of the file to save the program to.
File names can be up to 8 characters long and can contain only alphanumeric characters and underscores (_).
- If you want to specify a directory to save the file, include *DirectoryName* and a slash (/) followed by the file name.
If the directory does not exist, it will be created.
Directory names can be up to 8 characters long.
- If you omit the directory name, the file will be saved in the "SYSTEM" directory.
- A maximum of 15 files can be created in one directory.

◆Related topics

FILES (see p.26) KILL (see p.26) LOAD (see p.28)

•Example

SAVE "FILE01"

SAVE "USER/SAMPLE"

Saves the current program to the file "FILE01" in the "SYSTEM" directory.

Saves the current program as the file name "SAMPLE" in the "USER" directory.
If the "USER" directory does not exist, a new directory will be created.

◆Format

VIEW [*LineRange*]

Item	Notes
<i>LineRange</i>	Format: (a) <i>LineNumber</i> (b) <i>StartLine TO EndLine</i>
<i>StartLine</i>	format: { <i>LineNumber</i> FIRST LAST }
<i>EndLine</i>	format: { <i>LineNumber</i> FIRST LAST }

◆Explanation

- Display the program on the screen.
- You can specify the display range of a program by specifying a line range.
- *LineRange* can be specified in the following ways:
 - (a) Line number
If you specify *LineNumber*, that line will be displayed.
 - (b) *StartLine TO EndLine*
Displays lines from *StartLine* to *EndLine*.
FIRST means the first line of the program.
LAST means the last line of the program.
- If specifying *LineRange*, the end line must be the same as or later than the start line.
- If the program display does not fit on the screen, you can scroll the program display using the up and down cursor keys and the Page Up and Page Down keys.
- Pressing the Break or Enter key ends the display using the VIEW command.

◆Related topics

LIST (see p.27)

◆Example

VIEW	Show the entire program.
VIEW 200 TO 500	Displays lines 200 to 500 of the program.
VIEW FIRST TO 400	Displays lines 400 to 400 of the program.

Extended Commands

■ Graphics Specifications

◆ Screen Configuration

- The display area consists of a text screen and a graphics screen.

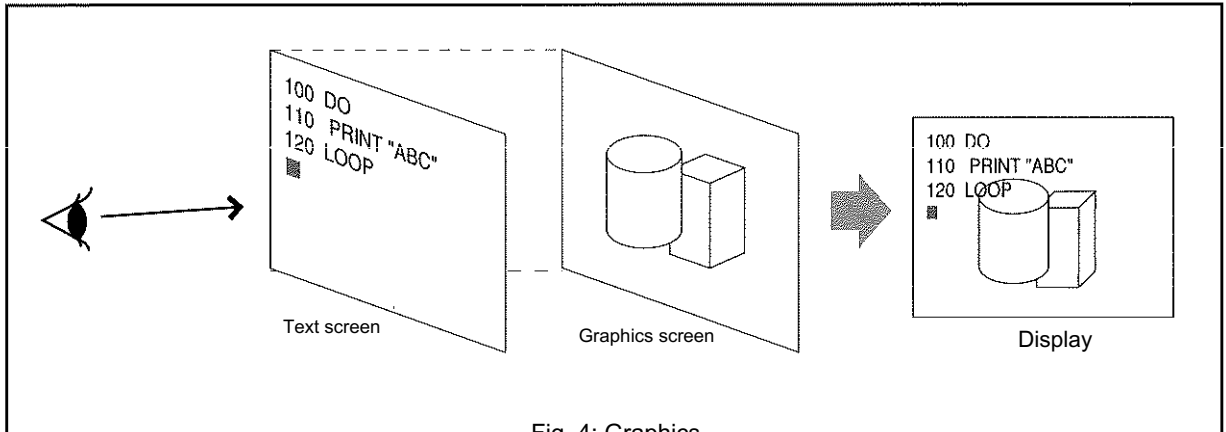


Fig. 4: Graphics Configuration

◆ Text Screen

- This is a screen for displaying text, and is always displayed in front of models and primitives.

◆ Graphics Screen

- The display area is 640 dots horizontally and 224 lines vertically.
- With the Z-buffer (see p.44), each pixel has a Z value.
- 256 gradations of RGB, 16,777,216 colors can be displayed.
- Each color is determined by combining the red, green, and blue components. Each component is specified by a value between 0 and 255. Specifying 0 makes the component the weakest, and 255 makes the component the strongest.
- You can specify transparency using the alpha value.
The alpha value is specified by a value between 0 and 128. 128 is fully opaque, and 0 is fully transparent.
- The pixels displayed on a television screen are twice as long vertically as they are horizontally.

◆ Color Specification

- The text color and mouse cursor color are specified by this color specification, rather than RGB.

■ Color Specification

Specification	色	Specification	色
RED	赤	BLUE	青
CYAN	水色	PURPLE	紫
YELLOW	黄	GREEN	緑
BLACK	黒	WHITE	白

- The text color is set using the TEXTCOL command (see p.40), and the mouse cursor color is set using the MOUCOL command (see p.36).

■ CLEAR

Clear text screen

◆Format

CLEAR

◆Explanation

- Clears all characters displayed on the text screen.
- Primitives and models are not affected.

■ DEVICE

Read/write device settings

◆Format

DEVICE *DeviceName*

◆Explanation

- Specify the device to read and write data from.
- In the *DeviceName* field, enter the name of the device to read from and write to. The device names are as follows:

Device list

Device Name	Device
MEMC1	"Playstation 2"(8MB) memory card connected to memory card port 1
MEMC2	"Playstation 2"(8MB) memory card connected to memory card port 2

- The default device is MEMC1.

◆Related topics

LOAD (see p.28)	SAVE (see p.30)	FILES (see p.26)
VARLOAD (see p.41)	VARSAVE (see p.41)	MODLOAD (see p.78)
TEXLOAD (see p.55)	PALLOAD (see p.50)	
BGMLOAD (see p.82)	SELOAD (see p.83)	

•Example

DEVICE MEMC2

Set the memory card connected to port 2 to be used for reading and writing of data.

◆Format

FSYNC *WaitTime*

Item	Note
<i>WaitTime</i> Range: 1 or more	

◆Explanation

- Pauses the program until the TV screen finishes displaying the content.
- *WaitTime* specifies the wait time for the program to stop.
The waiting time is measured from the time the last FSYNC command was executed until one screen drawing is completed.
- Usually, a wait time of 1 allows you to synchronize the display of a typical TV screen with the execution of your program.
- When the program finishes pausing, it continues execution from the next line.

◆Related topics

STOP (see p.24)

• Example/ Example to show effect of FSYNC

```

100 DO
110 CLEAR
120 PRINT "Snack!"
130 IF PAD(1,6)>0 THEN FSYNC 1
140 LOOP

```

<Explanation>

- 130** Executes the FSYNC command while the circle button is pressed. When the FSYNC command is not executing, the characters displayed on the screen will flicker, but if you hold down the O button, the characters displayed on the screen do not flicker because they are synchronized with the TV's display.

◆Format

INIT [ALL]

◆Explanation

- Initialize the EZ-BASIC environment.
- The items that will be initialized are the various settings for primitives, models, fog, cameras, effects, text screens, and sounds.
- Writing ALL will initialize the entire "EZ-BASIC" environment and load the initial setting data from the CD-ROM into various buffers.
- The program being edited will remain uninitialized.

■ LABEL

Specifying a label for a line

◆Format

(1) LABEL *LabelName*

(2) **LabelName*

◆Explanation

- Labels a line.
- When specifying a line number with a GOTO command or RESTORE command, you can use a label name instead of a line number. By adding an asterisk (*) before the label name, you can specify the line using the label name.
- It cannot be used to specify lines for direct commands such as LIST and RENUMBER.

◆Related topics

GOTO (see p.15)

ON~GOTO (see p.19)

RESTORE (see p.22)

IF THEN~END IF (see p.16)

•Example/ Using Labels

```
100 A=2
110 ON A GOTO *HOP,*STEP,*JUMP ELSE *EXIT Based on the value of A (2) execute from the line labeled by STEP
120 LABEL HOP
130 PRINT "HOP"
140 LABEL STEP
150 PRINT "STEP"
160 LABEL JUMP
170 PRINT "JUMP!"
180 LABEL EXIT
190 PRINT "END"
200 END
```

The command on line 110 moves execution to this line.

<Execution Result>

```
STEP
JUMP!
END
```

■ LOCATE

Move the text cursor

◆Format

LOCATE *TextX*, *TextY*

Item	Contents	Notes
<i>TextX</i>	Text x-coord.	Range: 0~37(TextMode 1), 0-49(TextMode 2)
<i>TextY</i>	Text y-coord	Range: 0~22

◆Explanation

- Moves the text cursor to the position specified by the text coordinates (*TextX*, *TextY*).
- Specifying a value outside the range of possible coordinates will result in an error.

•Example

```
LOCATE 30,20
```

Moves the cursor to text coordinates (30,20).

■MOUAREA

Specify the mouse cursor movement range

◆Format

MOUAREA *StartX,StartY,EndX,EndY*

Item	Contents	Note
<i>StartX</i>	Top left X coord.	Range: 0~639
<i>StartY</i>	Top left Y coord.	Range: 0~223
<i>EndX</i>	Lower right X coord.	Range: 0~639
<i>EndY</i>	Lower right Y coord.	Range: 0~223

◆Explanation

- Limits the range in which the mouse cursor can move to the rectangular area specified by (*StartX,StartY*) to (*EndX,EndY*) .
- An error occurs if *StartX>EndX* or *StartY>EndY* .

◆Related topics

MOUDISP (see p.36) MOUOFF (see p.37) MOUGET (see p.37)

•Example

MOUAREA 320,112,639,223 Limits the mouse cursor to 320~639 in the x 112~223 in the y direction.

■MOUCOL

Set the color of the mouse cursor

◆Format

MOUCOL *Color*

Item	Note
<i>Color</i>	(see p.32)

◆Explanation

- Sets the color of the mouse cursor.

◆Related topics

MOUAREA (see p.36) MOUDISP (see p.36)
MOUOFF (see p.37) MOUGET (see p.37)

•Example

MOUCOL RED Makes the mouse cursor red.

■MOUDISP

Displays the mouse cursor

◆Format

MOUDISP

◆Explanation

- Sets the mouse cursor to display

◆Related topics

MOUAREA (see p.36) MOUOFF (see p.37) MOUGET (see p.37)

■MOUGET

Read the mouse position and button state

◆Format

MOUGET *PosX*, *PosY*, *Button1*, *Button2*

Item	Contents	Note
<i>PosX</i>	Variable to store x coord.	
<i>PosY</i>	Variable to store y coord.	
<i>Button1</i>	Variable to store left button state	
<i>Button2</i>	Variable to store right button state	

◆Explanation

- If the left mouse button is pressed, the value of the variable passed as *Button1* will be 1, if the right mouse button is pressed, the value of *Button2* will be 1. If neither button is pressed, the value will be 0.
- The screen coordinates where the mouse cursor is currently displayed are assigned to the variables passed in *PosX* and *PosY*.

◆Related topics

MOUAREA (see p.36) MOUDISP (see p.36) MOUOFF (see p.37)

• Example

```
100 DO
110  MOUGET MX,MY,M1,M2           Assign current mouse position to MX,MY, and button states to M1,M2.
120  PRINT MX,MY,M1,M2
130  LOOP
```

■MOUOFF

Stop displaying mouse cursor

◆Format

MOUOFF

◆Explanation

- Stops displaying mouse cursor
- The cursor is not displayed, but the mouse state is still updated.

◆Related topics

MOUAREA (see p.36) MOUDISP (see p.36) MOUGET (see p.37)

■MOUPAT

Change the mouse cursor

◆Format

MOUPAT *Pat*

Item	Contents	Notes
<i>Pat</i>	Cursor pattern No.	Range: 1-18

◆Explanation

- Changes the shape of the mouse cursor to the pattern specified by *Pat*.

◆Format

MOUSET *MouseX,Mouse Y*

Item	Contents	Notes
<i>MouseX</i>	Cursor x coord.	Range 0-639
<i>MouseY</i>	Cursor y coord	Range 0-223

◆Explanation

- Moves the mouse cursor to the screen coordinates (*MouseX*, *MouseY*).

◆Related topics

MOUAREA (see p.36)

MOUDISP (see p.36)

MOUGET (see p.37)

MOUOFF (see p.37)

- **Example**

MOUSET 320,112

Move the cursor to the center of the screen

◆Format

ORDER *DrawingMode*

Item	Note
<i>DrawingMode</i>	Range 1-2

◆Explanation

- Sets the drawing order of models and primitives.
- A drawing mode of 1 draws models first, then primitives.
A drawing mode of 2 draws primitives first, then models.

◆Related topics

ZSORT(see p.81)

- **Example/** Example of switching drawing modes

```
100 MODDISP 1
```

110 TILEDISP 1

```
120 TILECOL 1,255,0,0,64
```

130 TILEPOS 1,200,50,440,224-50,800000 Place the primitive at a Z value of 800000.

140 CAMPOS 1,0,0,-50

150 DO

160 CLEAR

```
170 PRINT "PRESS START button"
```

180 D=PAD(1,20)+1

190 ORDER D

200 FSYNC 1

```
210 LOOP
```

Pressing the START button will set the drawing mode to 2.
Releasing the START button will set the drawing mode to 1.

◆Format

SCREEN *ScreenMode*

◆Explanation

- Sets how the screen is split.
- Depending on the screen mode, the following screen divisions are possible:

Screen Mode Types

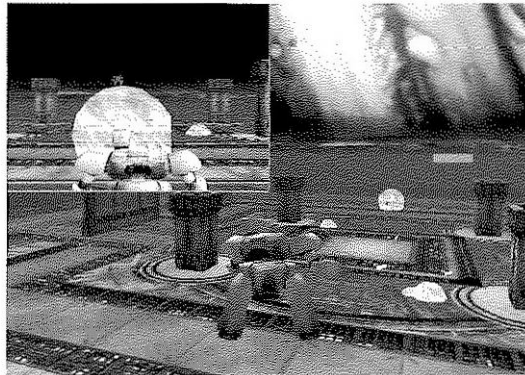
Screen Mode	Effect
<i>SINGLE</i>	Single fullscreen display. The image from camera number 1 will be displayed.
<i>SPLIT</i>	The screen is split into two halves, left (view of camera 1) and right (view of camera 2).
<i>VERTICAL</i>	The screen is split into two halves, top (view of camera 1) and bottom (view of camera 2).
<i>WINDOW StartX, StartY, EndX, EndY</i>	The image captured by camera number 2 will be displayed in the rectangular area on the screen specified by (StartX, StartY) - (EndX, EndY).

- Split screen affects the display of models, not primitives and text.

• Example

SCREEN WINDOW 0,0,300,120

The image captured by camera number 2 is displayed in a rectangular area on the screen captured by camera 1, with screen coordinates (0,0) at the top left and (300,120) at the bottom right (top left quarter of screen).



■ TEXTCOL

Set the color of text

◆Format

TEXTCOL *Color*

Item	Note
<i>Color</i>	(see p.32)

◆Explanation

- Sets the color of displayed text.

◆Related topics

PRINT (see p.20)

•Example

TEXTCOL BLUE

Text is set to display in blue.

■ TEXTMODE

Set text display mode

◆Format

TEXTMODE *Mode*

Item	Note
<i>Mode</i>	Range: 1-2

◆Explanation

- Set the number of characters to be displayed on the text screen.
- The *Mode* value allows you to set the number of characters that can be displayed on the text screen.

Text Mode

<i>Mode</i>	Displayed characters
1	38 ch. wide, 23 ch. high
2	50 ch. wide, 23 ch. high

- Currently displayed text is cleared when the textmode is changed.

•Example

TEXTMODE 2

Sets the text mode to display 50 by 25 characters.

■VARLOAD

Load variable data from file

◆Format

VARLOAD [*DirectoryName* /] *FileName*, *VariableName*

◆Explanation

- Read data from a file and assign it to a variable.
- The file can be a numeric variable file with the extension VRN or a string variable file with the extension VRS.
- You can also specify an array as a variable.
- If the type of data to be read does not match the type of the variable, an error will occur.
- If you specify a directory name, the files in that directory will be read.
If the directory name is omitted, the file is read from the "SYSTEM" directory.

◆Related topics

VARSAVE (see p.41)

•Example

VARLOAD "SAVE01",DATA01

Loads the data in the file "SAVE01" in the "SYSTEM" directory into the variable DATA01.

■VARSAVE

Save variable data to file

◆Format

VARSAVE [*DirectoryName* /] *FileName*, *VariableName*

◆Explanation

Writes the value of a variable to a file.

You can also specify an array as a variable. In the case of an array, specify the array variable name.

In this case, all the values of the array elements are saved to the file.

If you specify a numeric variable, the saved file will have extension VRN, and a string variable will have extension VRS.

If you want to save a file in a directory, write the directory name and a slash (/) followed by the file name.

If the specified directory does not exist, it will be created.

Directory and file names can contain up to eight alphanumeric characters and underscores (_).

If you omit the directory name, the file will be saved in the "SYSTEM" directory.

You can create up to 15 files in one directory.

◆Related topics

VARLOAD (see p.41)

• Example 1

VARSAVE "SAVE/DATA1",A

VARSAVE "DATA",B

Save the value of variable A to the file "DATA1" in the "SAVE" directory.
Variable B is an array. In this case, save all elements of the array to the file "DATA2" in the "SYSTEM" directory.

• Example 2

```
100 SAV01=234
110 PRINT "Would you like to save?"
120 DO
130 IF PAD(1,6)>0 THEN
140 VARSAVE "SAVE01",SAV01
150 PRINT "Save completed."
160 EXIT DO
170 ELSEIF PAD(1,7)>0 THEN
180 PRINT "Save canceled."
190 EXIT DO
200 ENDIF
210 LOOP
220 END
```

If the circle button is pressed
Save the value of SAV01 to file "SAVE01"

If X is pressed, cancel.

◆Format

VIBRATE *ContNo*, *VibrateL*, *VibrateR*, *Time*

Item	Contents	Note
<i>ContNo</i>	Controller number	Range: 1-2
<i>VibrateL</i>	Left motor	Range: 0-255
<i>VibrateR</i>	Right motor	Range: 0-1
<i>Time</i>	Vibrate time(sec)	Range: 0-10

◆Explanation

- Vibrate the analog controller (DUALSHOCK 2).
 - In *ContNo*, specify the number of the controller port to which the controller you want to vibrate is connected.
 - The analog controller (DUALSHOCK 2) has two motors with different characteristics built in, one on the left and one on the right, and you can control the vibration of each.
The *VibrateL* value specifies the strength of the vibration of the left vibration motor.
The *VibrateR* value specifies whether the right vibration motor should be stopped or vibrated.
 - Specify the vibration duration in *Time*.
To stop the vibration, set *Time* to 0.
 - If you specify a value outside the range for each value, an error will occur.
- *Please avoid prolonged vibration as this may shorten the life of the controller.

•Example

VIBRATE 1,255,1,5 Strongly vibrate the controller connected to port 1 for 5 seconds.

2D Graphics Related

■ Screen description

◆ Screen coordinate system

The screen coordinates as they appear on the TV.

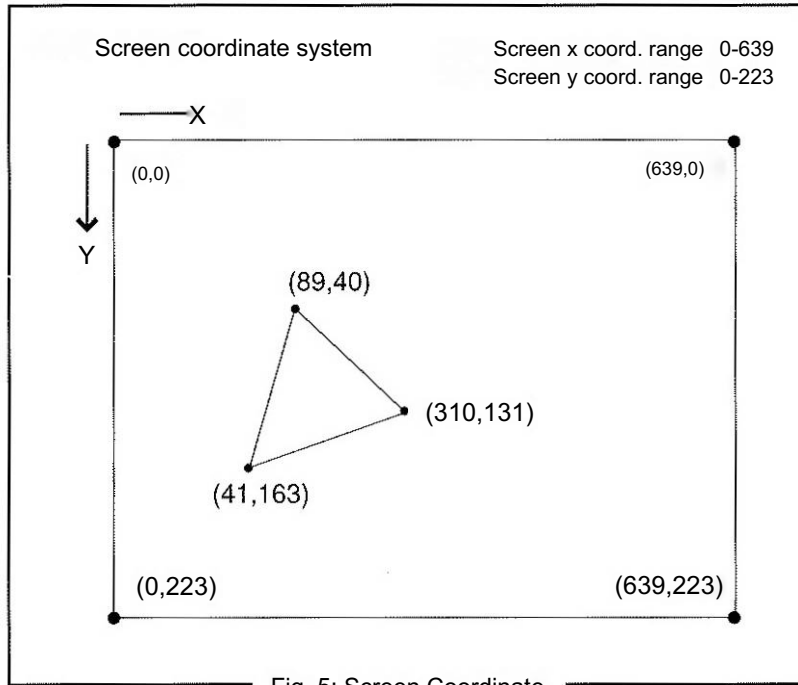


Fig. 5: Screen Coordinate System

◆ Primitives

- Primitives are graphics elements used as the fundamental units of drawing. There are four types of primitives: line, tile, triangle, and sprite.
- The front-to-back ordering of primitives is determined by the Z value.
The Z value can be specified between 0 and 16777200. 0 is placed at the back, and 16777200 at the front when drawing.
- Each primitive is usually drawn in primitive number order.
- No camera, light or fog effects from 3D graphics functions influence 2D primitives.

◆ Text

- When the text mode (see p.40) is 1, the maximum number of characters that can be displayed on the screen is 38 characters wide and 23 characters high. When the text mode is 2, the screen is 50 characters wide and 23 characters high.
- Full-width kana and kanji characters require a horizontal display width equivalent to two half-width characters per character.
- The Z value cannot be set. It is always displayed in front of the screen.
- No camera, light or fog effects from 3D graphics functions influence text display.

◆ Mouse cursor

- When a USB mouse is connected, the mouse cursor can be manipulated. You can switch the display / non-display in the program and limit the cursor movement range.
- No camera, light or fog effects from 3D graphics functions influence mouse cursor display.

■Z-buffer Method

- This is the process by which each pixel that makes up the screen has a depth value (Z-value). Having the z-value enables determining the front-to-back order of two primitives and draw them appropriately.

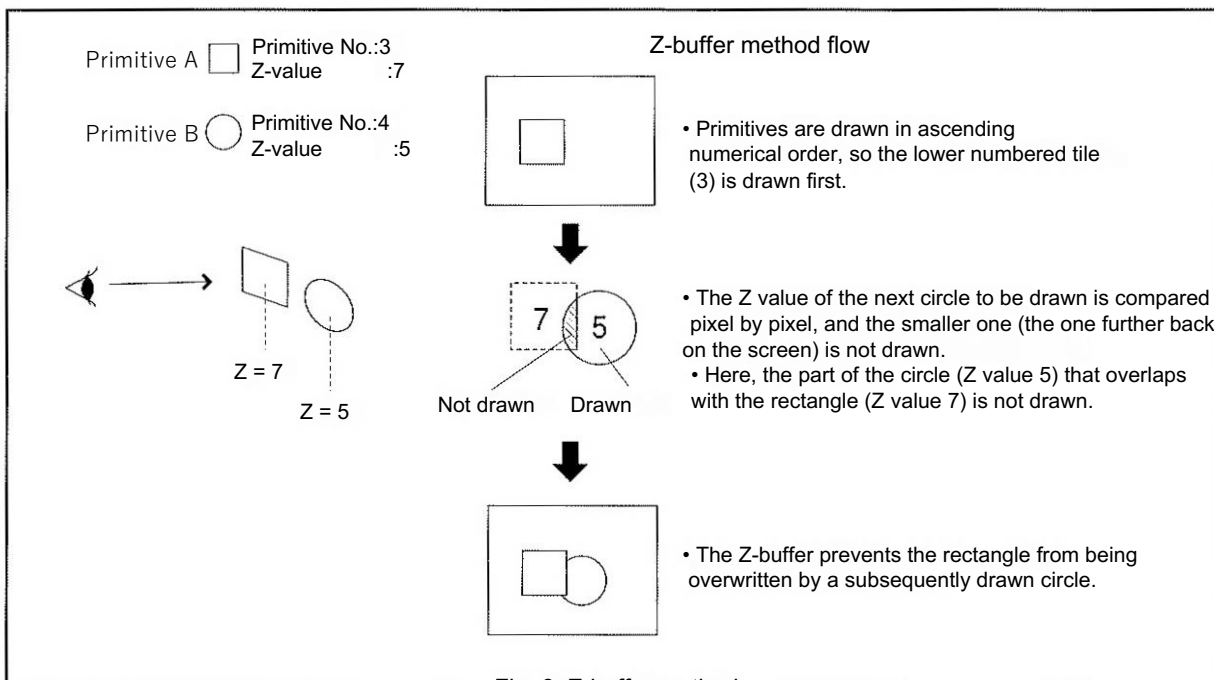


Fig. 6: Z-buffer method

■Z-buffer issues

- If you are using transparent models, you may encounter Z-buffer issues. This refers to when the overlapping area between a transparent model in the foreground and an object in the background is not drawn due to the Z-buffer function, so nothing is drawn in the overlapping semi-transparent area. To alleviate this problem, use Z-sorting (see p.45).

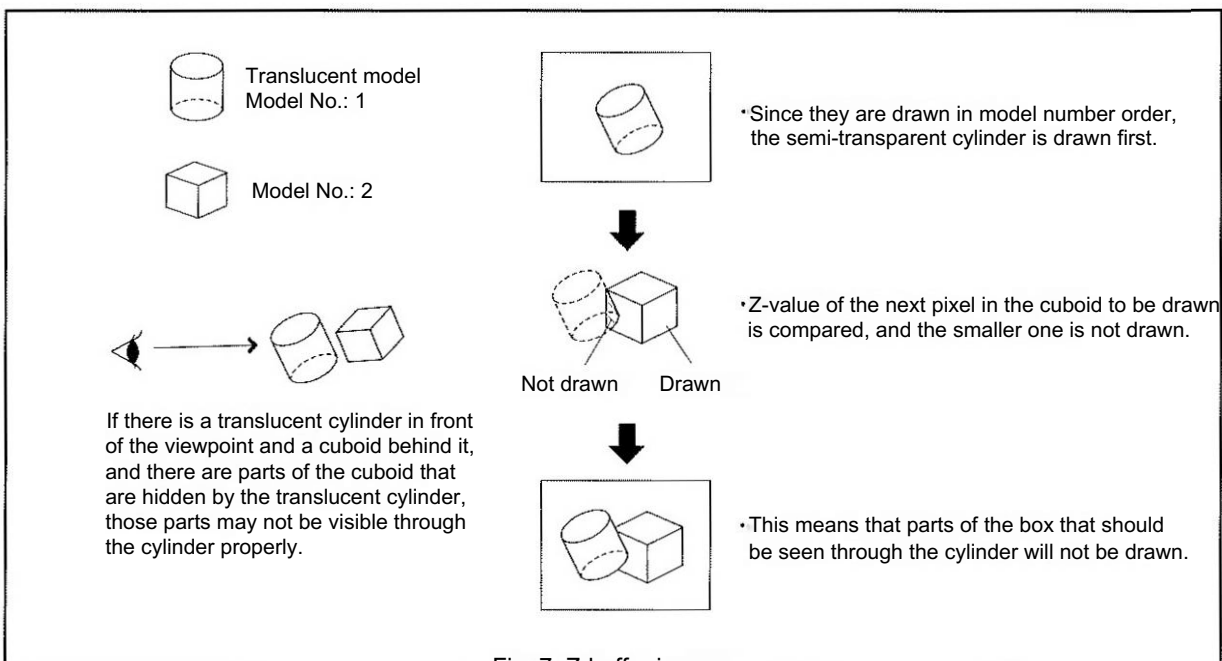


Fig. 7: Z-buffer issues

- By sorting models by Z value, you can determine the front-to-back relationship in advance, allowing overlapping semi-transparent areas to be drawn appropriately.
- Z-sorting can largely avoid the Z-buffer problem, but on the other hand, Z-sorting increases the computational load.
- Z sorting is performed based on the model's origin (not per point), so depending on the shape of the model, the Z buffer problem may not be completely resolved, and the model may not be drawn properly.
- To perform Z sorting, set the model to be sorted using the ZSORT command (see p.81).

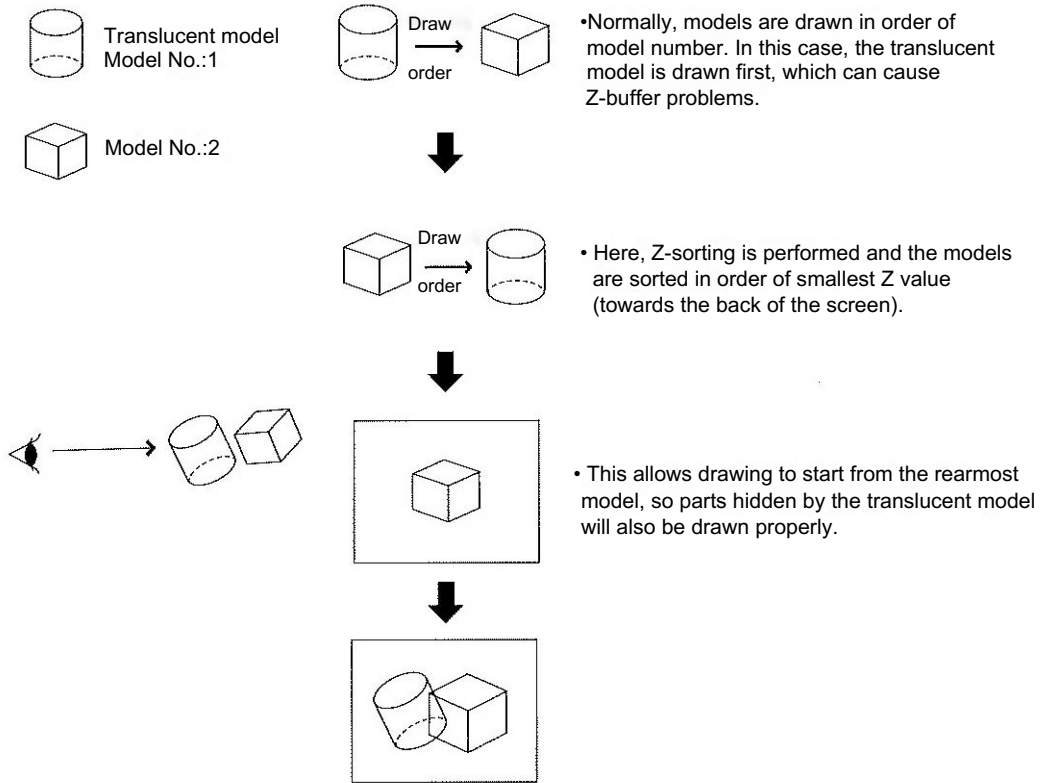


Fig. 8: Z-sort

■BGCOL

Set background color

◆Format

BGCOL *Red,Green,Blue*

Item	Content	Note
<i>Red</i>	Red intensity	Range:0-255
<i>Green</i>	Green intensity	Range:0-255
<i>Blue</i>	Blue intensity	Range:0-255

◆Explanation

- Set the background color of the screen.
- Set the color components for each value of Red, Green, and Blue.
- If you specify a value outside the range for each value, an error will be thrown.

•Example

BGCOL 255,0,0	The background color will change to red.
BGCOL 0,255,255	The background color will change to light blue.
BGCOL 200,133,43	The background color will change to orange.

■BSCOL

Set screen color when ScrollLock enabled

◆Format

BSCOL *Red,Green,Blue,Alpha*

Item	Contents	Note
<i>Red</i>	Red intensity	Range:0-255
<i>Green</i>	Green intensity	Range:0-255
<i>Blue</i>	Blue intensity	Range:0~255
<i>Alpha</i>	Alpha value	Range:0-128

◆Explanation

- Set the screen color when ScrollLock is enabled.
- You can enable Scroll Lock by pressing the Scroll Lock key.
In this state, you can release Scroll Lock by pressing the Scroll Lock key again.
- Specify the color with *Red*, *Green*, and *Blue*, and specify the transparency with *Alpha*.

•Example

BSCOL 128,0,0,96	Pressing the Scroll Lock key will turn the screen color semi-transparent red.
------------------	---

◆Format

EFFECT *Alpha, StartX, StartY, EndX, EndY, Z*

Item	Contents	Note
<i>Alpha</i>	Alpha value	Range:0~127
<i>StartX</i>	Start X coordinate	Range:-1728~2367
<i>StartY</i>	Start Y coordinate	Range:-1936~2159
<i>EndX</i>	End X Coordinate	Range:-1728~2367
<i>EndY</i>	End Y Coordinate	Range:-1936~2159
<i>Z</i>	Depth value	Range:0~16777200

◆Explanation

- Set the drawn image to remain displayed until the next drawing. Varying based on the settings, an afterimage effect on the screen is created.
- Alpha specifies the rate at which the drawn image fades. The higher this value, the slower the image fades, resulting in a stronger afterimage effect.
- The drawn image is converged to the area specified by the screen coordinates (StartX, StartY) to (EndXEndY) . If you take a larger image than the actual width, it will expand, and if you take a smaller image, it will shrink.
- Setting Alpha to 0 will disable the effect of the EFFECT command.

•Example

EFFECT 60,0,0,640,224,0	A weak afterimage effect is achieved.
EFFECT 110,0,0,640,224,0	A strong afterimage effect is achieved.
EFFECT 110,-8,-4,648,228,0	Creates an expanding afterimage effect.
EFFECT 0,0,0,0,0,0	Disable the effect.

◆Format

LINECOL *LinePrimNo, Red, Green, Blue, Alpha*

Item	Contents	Notes
<i>LinePrimNo</i>	Primitive number	Range: 1~1000
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255
<i>Alpha</i>	Alpha value	Range: 0~128

◆Explanation

- Set the color of the line primitive .
- Specify the number of the line primitive to which you want to apply the LINECOL command in LinePrimNo .
- For each of the Red, Green, and Blue values, 0 is the weakest and 255 is the strongest intensity.
- Alpha is closer to 0 for transparent and closer to 128 for opaque.

◆Related Topics

LINEDISP (see p.48) LINEOFF (see p.48) LINEPOS (see p.49)

• Example

LINECOL 1,0,255,0,128	Set the line color of line primitive number 1 to green.
LINECOL 2,35,200,40,64	Set the line of line primitive number 2 to a semi-transparent yellow-green color.

■ LINEDISP

Display line primitive

◆ Format

LINEDISP *LinePrimNo*

Item	Content	Note
<i>LinePrimNo</i>	Primitive number	Range: 1-1000

◆ Explanation

- Displays the line primitive specified by *LinePrimNo*.

◆ Related topics

LINEOFF (see p.48) LINEPOS (see p.49)

• Example

LINEDISP 55 Displays line primitive number 55.

■ LINEOFF

Hide line primitive

◆ Format

LINEOFF *LinePrimNo*

Item	Contents	Note
<i>LinePrimNo</i>	Primitive number	Range: 1-1000

◆ Explanation

- Hides the line primitive specified by *LinePrimNo*.
- Properties can still be updated by commands like LINEPOS even after LINEOFF is executed.

◆ Related topics

LINECOL (see p.47) LINEDISP (see p.48) LINEPOS (see p.49)

• Example

LINEOFF10 Hide line primitive 10.

◆Format

LINEPOS *LinePrimNo,StartX,StartY,EndX,EndY,Z*

Item	Contents	Note
<i>LinePrimNo</i>	Primitive number	Range: 1~1000
<i>StartX</i>	Start point x-coord.	Range: -1728~2367
<i>StartY</i>	Start point y-coord.	Range: -1936~2159
<i>EndX</i>	End point x-coord.	Range: -1728~2367
<i>EndY</i>	End point y-coord.	Range: -1936~2159
<i>Z</i>	Z-value	Range: 0~16777200

◆Explanation

- Set the display position of a line primitive.
- Specify the number of the line primitive for which you want to set the display position in *LinePrimNo*.
- Draws a line connecting the screen coordinate values (*StartX, StartY*) and (*EndX, EndY*).
- Specify the Z value that indicates the depth of the line primitive. The closer Z is to 0, the further back it is placed, and the closer it is to 16777200, the closer it is to the front.

◆Related topics

LINEDISP (see p.48) LINEOFF (see p.48)

• Example

LINEPOS 1,0,0,640,224,0 Sets line primitive number 1 to go from (0,0) to (640,224), at the back of the screen.

◆Format

PALCOPY *DstPal.SrcPal*

Item	Contents	Note
<i>DstPal</i>	Copy destination palette	Range: 1~32
<i>SrcPal</i>	Copy source palette	Range: 1~32

◆Explanation

- Copy palette data to another palette.

◆Related topics

PALLOAD (see p.50)

• Example

PALCOPY 1,5 Copy the palette data from number 5 to palette number 1.

■PALLOAD

Loading palette data

◆Format

PALLOAD [*DirectoryName* /] *FileName*

◆Explanation

- Reads PAL format palette data from a file in the specified directory.
- If the directory name is omitted, the file will be read from the "SYSTEM" directory.
- When the PALLOAD command is executed, all palette data saved in the file is loaded.
- When new palette data is loaded from a file, the previously used palettes will be overwritten.
However, any unused palette numbers in the loaded file will retain their current data.

• Example

PALLOAD "PAL10" Loads the palette data saved in the file name "PAL10" in the "SYSTEM" directory.

■SPRDISP

Display sprite primitive

◆Format

SPRDISP *SpritePrimNo*

Item	Content
<i>SpritePrimNo</i>	range:1~1000

◆Explanation

- Displays the sprite primitive specified by *SpritePrimNo*.

◆Related topics

SPROFF (see p.50)	SPRPAL (see p.51)	SPRPOS (see p.52)
SPRPOS2 (see p.53)	SPRTEX (see p.54)	

• Example

SPRDISP 66 Display sprite primitive number 66.

■SPROFF

Hide sprite primitive

◆Format

SPROFF *SpritePrimNo*

Item	Content	Note
<i>SpritePrimNo</i>	Primitive number	Range: 1~1000

◆Explanation

- Hides the sprite primitive specified by *SpritePrimNo*.
- Properties can still be updated by commands like SPRPOS even after SPROFF is executed.

◆Related topics

SPRDISP (see p.50)	SPRPAL (see p.51)	SPRPOS (see p.52)
SPRPOS2 (see p.53)	SPRTEX (see p.54)	

• Example

SPROFF 66 Hide sprite primitive 66.

◆Format

SPRPAL *SpritePrimNo,PaletteNo,PaletteOfs,AlphaMode*

Item	Content	Note
<i>SpritePrimNo</i>	Primitive number	Range:1-1000
<i>PaletteNo</i>	Palette number	Range:1-32
<i>PaletteOfs</i>	Palette offset value	Range:0-15
<i>AlphaMode</i>	Alpha mode	Mode: 0:subtract mode 1:Normal mode 2:add mode

◆Explanation

- Sets the palette used by the sprite primitive *SpritePrimNo*.
- *PaletteNo* specifies the number of the palette to apply to the sprite primitive.
- If the image you are using is drawn in 16-color (4-bit) mode, specify the position of the color used in *PaletteOfs*.

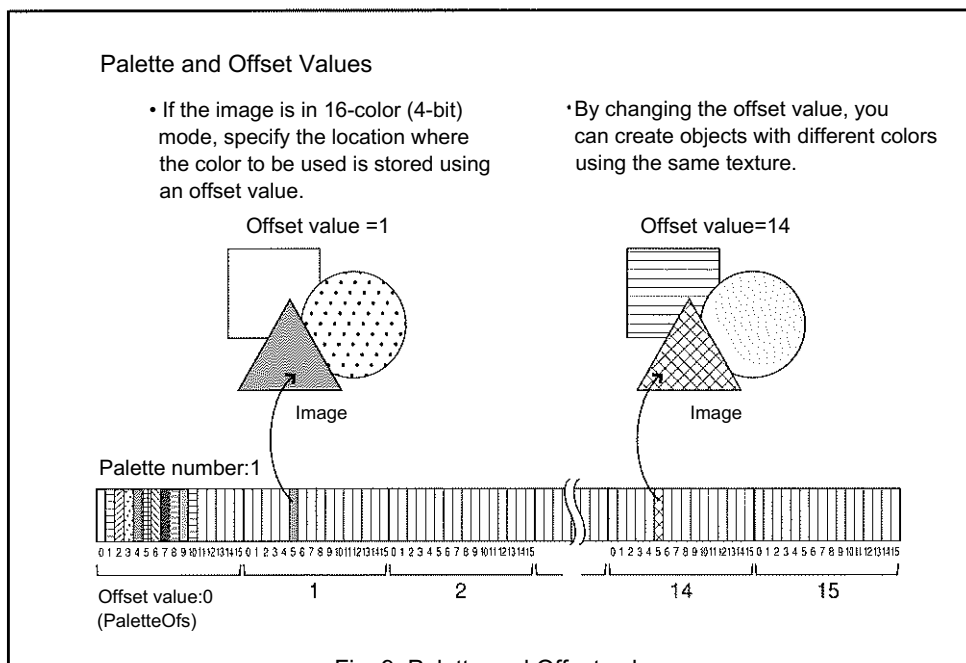
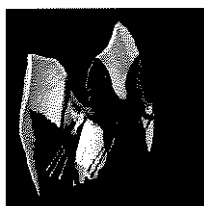
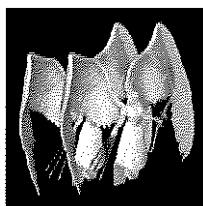


Fig. 9: Palette and Offset values

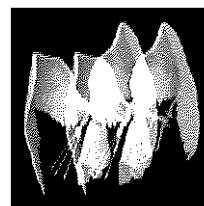
- If the image used is in 256-color (8-bit) mode, set *PaletteOfs* to 0. With any other value, appropriate color will not be used.
- *AlphaMode* sets the calculation method for semi-transparent colors.



Subtract (AlphaMode=0)



Normal (AlphaMode=1)



Add (AlphaMode=2)

◆Related topics

SPRDISP (see p.50)

SPROFF (see p.50)

SPRPOS (see p.52)

SPRPOS2 (see p.53)

SPRTEX (see p.54)

• Example

SPRPAL 1,5,3,1 Sets sprite primitive 1 to the color of palette offset value 3 from palette 5 in normal alpha mode.

◆Format

SPRPOS *SpritePrimNo*, *ScreenX*, *ScreenY*, *Z*

Item	Contents	Note
<i>SpritePrimNo</i>	Primitive number	Range: 1-1000
<i>ScreenX</i>	Screen x coordinate	Range: -1728-2367(※)
<i>ScreenY</i>	Screen y coordinate	Range: -1936-2159(※)
<i>Z</i>	Depth value	Range: 0-16777200

※This value includes the horizontal and vertical size of the sprite.

◆Explanation

- Sets the display position of the sprite primitive specified by *SpritePrimNo*.
- The size of the sprite will be the size of the texture data specified by the SPRTEX command (see p. 54).
- The sprite primitive is placed with the top left corner at the screen coordinates (*ScreenX*, *ScreenY*).
- Specify the Z-value that indicates the depth of the sprite primitive in argument Z. The closer Z is to 0, the further back it will be placed, and the closer it is to 16777200, the closer to the screen plane it will be placed.
- Specifying a value outside of the range for any argument will result in an error.
- The sprite will not be displayed on the screen unless the SPRDISP command (see p.50) is executed.

◆Related topics

SPRDISP (see p.50) SPROFF (see p.50) SPRPAL (see p.51)
 SPRPOS2 (see p.53) SPRTEX (see p.54)

• Example/ Moving a sprite using controller input

```

100 SPRDISP 1
110 DO
120  X=PAD(1,14)           From the controller input, variable X and
130  Y=PAD(1,13)           variable Y are assigned values.
140  SPRPOS 1,X,Y,0        Places the sprite on the screen coordinates (X, Y).
150  FSYNC 1
160 LOOP
  
```

◆Format

SPRPOS2 *SpritePrimNo*, *Start X*, *StartY*, *EndX*, *EndY*, *Z*

Item	Content	Note
<i>SpritePrimNo</i>	Primitive number	Range: 1~1000
<i>StartX</i>	Start point x coord.	Range: -1728~2367
<i>StartY</i>	Start point y coord.	Range: -1936~2159
<i>EndX</i>	End point x coord.	Range: -1728~2367
<i>EndY</i>	End point y coord.	Range: -1936~2159
<i>Z</i>	Depth value	Range: 0~16777200

◆Explanation

- Sets the display position of the sprite primitive specified by *SpritePrimNo*.
When placing a sprite primitive, you can specify the area to place it in. This allows you to scale or flip the sprite.
- The sprite primitive is enlarged or reduced to fit within the area specified by the screen coordinates (*StartX*, *StartY*) to (*EndX*, *EndY*) .
- If *StartX*>*EndX*, the sprite primitive will be placed flipped horizontally.
If *StartY*>*EndY*, the sprite primitive will be placed flipped vertically.
- Specify the Z-value that indicates the depth of the sprite primitive in argument Z. The closer Z is to 0, the further back it will be placed, and the closer it is to 16777200, the closer to the screen plane it will be placed.
- If you specify a value outside the range, an error will occur.
- It will not be displayed on the screen unless the SPRDISP command (see p. 50) is executed.

◆Related topics

SPRDISP (see p.50) SPROFF (see p.50) SPRPAL (see p.51)
SPRPOS (see p.52) SPRTEX (see p.54)

• Example/ Setting sprite scaling by controller input

```

100 SPRDISP 1
110 D0
120 X1=PAD(1,14)           Assign each value to position the sprite from the controller input .
130 Y1=PAD(1,13)
140 X2=PAD(1,16)
150 Y2=PAD(1,15)
160 SPRPOS2 1,X1,Y1,X2,Y2,0  Places the sprite at the specified position.
170 PRINT X1,Y1,X2,Y2      Display the placement position for confirmation.
180 FSYNC 1
190 LOOP

```

◆Format

SPRTEX *SpritePrimNo*, *TexPageNo*, *StartTX*, *StartTY*, *EndTX*, *EndTY*

Item	Contents	Note
<i>SpritePrimNo</i>	Primitive number	Range: 1~1000
<i>TexPageNo</i>	Texture page number	Range: 1~16
<i>StartTX</i>	Texture start point x coord.	Range: 0~512 (16 color(4bit) mode) 0~256 (256 color(8bit) mode)
<i>StartTY</i>	Texture start point y coord.	Range: 0~256
<i>EndTX</i>	Texture endpoint x coord.	Range: 0~512 (16 color(4bit) mode) 0~256 (256 color(8bit) mode)
<i>EndTY</i>	Texture endpoint y coord.	Range: 0~256

◆Explanation

- Specifies image data to be used for a sprite primitive.
- *TexPageNo* specifies the number of the texture page to use.
The data used for the texture page is loaded using the TEXLOAD command (see p. 55).
- The rectangular area specified by the texture coordinates (*StartTX*, *StartTY*) to (*EndTX*, *EndTY*) is set as a sprite primitive.

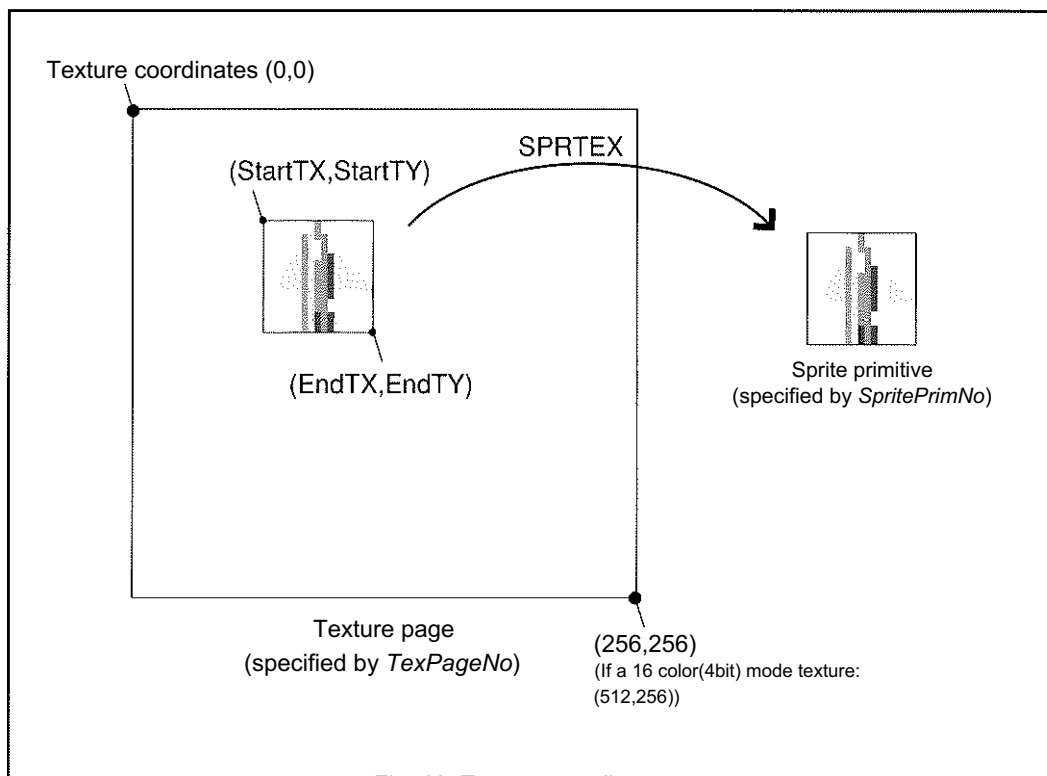


Fig. 10: Texture coordinate system

◆Related topics

SPRDISP (see p.50)
SPRPOS (see p.52)

SPROFF (see p.50)
SPRPOS2 (see p.53)

SPRPAL (see p.51)

• Example

SPRTEX 2,0,16,64,32

For sprite primitive 1, texture coordinates from start point (0,16) to end point (64,32) on texture page 2 are assigned.

◆Format

TEXCOPY *DstPage,SrcPage*

Item	Content	Note
<i>DstPage</i>	Copy destination tex page number	Range: 1~16
<i>SrcPage</i>	Copy source tex page number	Range: 1~16

◆Explanation

- Copy the contents of a texture page to another texture page.

◆Related topics

TEXLOAD (see p.55)

• Example

TEXCOPY 1,5 Copy the contents of texture page 5 to texture page 1.

◆Format

TEXLOAD *DirectoryName / FileName*

◆Explanation

- Texture data is read from the TEX format file specified by *FileName* and stored in the texture page.
- If you specify *DirectoryName*, data will be read from the files in that directory.
If you omit *DirectoryName*, the "SYSTEM" directory will be used.
- The texture page consists of 16 pages, and page numbers 1 to 16 are assigned sequentially from the top.
- When new texture data is loaded from a file, the texture data that was previously in use is overwritten.
However, texture pages that do not have texture data to load will retain their current data.

◆Related topics

SPRTEX (see p.54)

• Example

TEXLOAD "TEX01" Loads texture data from the file named "TEX01" in the "SYSTEM" directory into the texture page.

TILECOL

Set tile primitive color

◆Format

TILECOL *TilePrimNo,Red,Green,Blue,Alpha*

Item	Content	Note
<i>TilePrimNo</i>	Primitive number	Range: 1~1000
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255
<i>Alpha</i>	Alpha value	Range: 0~128

◆Explanation

- Sets the color of the tile primitive specified by *TilePrimNo*.
- For each of the Red, Green, and Blue values, 0 is the weakest and 255 is the strongest intensity.
- Alpha closer to 0 is more transparent, and closer to 128 is more opaque.
- If you specify a value outside the range for each value, an error will occur.

◆Related topics

TILEDISP (see p.56) TILEOFF(see p.57) TILEPOS (see p.57)

• Example

```
TILECOL 1,0,255,0,128      Set tile primitive one to green.
TILECOL 4,0,64,32,64      Set tile primitive 4 to semi-transparent dark green.
```

TILEDISP

Display tile primitive

◆Format

TILEDISP *TilePrimNo*

Item	Content	Note
<i>TilePrimNo</i>	Primitive number	Range: 1~1000

◆Explanation

- Display the tile primitive specified by *TilePrimNo*.

◆Related topics

TILEOFF (see p.57) TILEPOS (see p.57)

• Example

```
TILEDISP 77      Display tile primitive number 77.
```

■TILEOFF

Hide tile primitive

◆Format

TILEOFF *TilePrimNo*

Item	Content	Note
<i>TilePrimNo</i>	Primitive number	Range: 1~1000

◆Explanation

- Hides the tile primitive specified by *TilePrimNo*.
- Properties can still be updated by commands like TILEPOS even after TILEOFF is executed.

◆Related topics

TILEDISP (see p.56) TILEPOS (see p.57)

•Example

TILEOFF 88 Hides tile primitive 88

■TILEPOS

Set display position of tile primitive

◆Format

TILEPOS *TilePrimNo, StartX, StartY, EndX, EndY, Z*

Item	Content	note
<i>TilePrimNo</i>	Primitive number	Range: 1~1000
<i>StartX</i>	Start point x coord.	Range: -1728~2367
<i>StartY</i>	Start point y coord.	Range: -1936~2159
<i>EndX</i>	End point x coord.	Range: -1728~2367
<i>EndY</i>	End point y coord.	Range: -1936~2159
<i>Z</i>	Depth value	Range: 0~16777200

◆Explanation

- Sets the display position of the tile primitive specified by *TilePrimNo*.
- Specifies the rectangular area to place the tile from screen coordinates (*StartX, StartY*) to (*EndX, EndY*).
- Specify the Z-value that indicates the depth of the tile primitive in argument Z. 0 is furthest back, 16777200 is furthest forward.
- It will not be displayed on the screen unless the TILEDISP command (see p.56) is executed.

◆Related topics

TILEDISP (see p.56) TILEOFF (see p.57)

• Example

TILEPOS 1, 0, 0, 640, 224, 0 Sets tile primitive 1 to the back of the screen with the size indicated from the start point (0,0) to the end point (640,224)

◆Format

TRICOL *TrianglePrimNo,Red,Green,Blue,Alpha*

Item	Content	Note
<i>TrianglePrimNo</i>	Primitive number	Range: 1~1000
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255
<i>Alpha</i>	Alpha value	Range: 0~128

◆Explanation

- Sets the color of the triangle primitive specified by *TrianglePrimNo*.
- For each of the Red, Green, and Blue values, 0 is the weakest and 255 is the strongest intensity.
- Alpha closer to 0 is more transparent, and closer to 128 is more opaque.
- If you specify a value outside the range for each value, an error will occur.

◆Related topics

TRICOL3 (see p.59) TRIDISP (see p.56)
 TRIOFF (see p.60) TRIPOS (see p.60)

• Example

TRICOL 3,255,255,255,128 Set triangle primitive 3 to white.
 TRICOL 4,0,64,32,64 Set triangle primitive 4 to semi-transparent cyan.

■TRICOL3

Set color of each vertex of triangle primitive

◆Format

TRICOL3 *TrianglePrimNo,Red1,Green1,Blue1,Alpha1,Red2,Green2,Blue2,Alpha2,Red3,Green3,Blue3,Alpha3*

Item	Content	Note
<i>TrianglePrimNo</i>	Primitive number	Range: 1~1000
<i>Red1</i>	Vertex 1 Red intensity	Range: 0~255
<i>Green1</i>	Vertex 1 Green intensity	Range: 0~255
<i>Blue1</i>	Vertex 1 Blue intensity	Range: 0~255
<i>Alpha1</i>	Vertex 1 Alpha value	Range: 0~128
<i>Red2</i>	Vertex 2 Red intensity	Range: 0~255
<i>Green2</i>	Vertex 2 Green intensity	Range: 0~255
<i>Blue2</i>	Vertex 2 Blue intensity	Range: 0~255
<i>Alpha2</i>	Vertex 2 Alpha value	Range: 0~128
<i>Red3</i>	Vertex 3 Red intensity	Range: 0~255
<i>Green3</i>	Vertex 3 Green intensity	Range: 0~255
<i>Blue3</i>	Vertex 3 Blue intensity	Range: 0~255
<i>Alpha3</i>	Vertex 3 Alpha value	Range: 0~128

◆Explanation

- Sets the color of the triangle primitive specified in *TrianglePrimNo* for each vertex.
The inside of the triangle is filled with a gradient of the color of each vertex.
- For each color component, 0 is the weakest and 255 is the strongest.
- For each alpha value, the closer it is to 0, the more transparent it is, and the closer it is to 128, the more opaque it is.
- If you specify a value outside the range for each value, an error will occur.

◆Related topics

TRICOL (see p.58) TRIDISP (see p.59)
TRIOFF (see p.60) TRIPOS (see p.60)

• Example

TRICOL3 255,255,255,128, 0,255,255,64,255,0,0,34 For triangle primitive number 3, set the color of vertex 1 to white, the color of vertex 2 to light blue with semi-transparent color, and the color of vertex 3 to red with transparency 34.

■TRIDISP

Display triangle primitive

◆Format

TRIDISP *TrianglePrimNo*

Item	Content	Note
<i>TrianglePrimNo</i>	Primitive number	Range: 1~1000

◆Explanation

- Display the triangle primitive specified by *TrianglePrimNo*.

◆Related topics

TRIOFF (see p.60) TRIPOS (see p.60)

• Example

TRIDISP 99 Display triangle primitive 99.

■TRIOFF

Hide triangle primitive

◆Format

TRIOFF *TrianglePrimNo*

Item	Content	Item
<i>TrianglePrimNo</i>	Primitive number	Range: 1~1000

◆Explanation

- Hide the triangle primitive specified by *TrianglePrimNo*.
- Properties can still be updated by commands like TRIPOS even after TRIOFF is executed.

◆Related topics

TRICOL (see p.58)	TRICOL3 (see p.59)
TRIDISP(see p.59)	TRIPOS (see p.60)

•Example

TRIOFF 101 Hide triangle primitive 101

■TRIPOS

Set display position of tile primitive

◆Format

TRIPOS *TrianglePrimNo, ScreenX1,ScreenY1,ScreenX2,ScreenY2,ScreenX3,ScreenY3,Z*

Item	Content	Note
<i>TrianglePrimNo</i>	Primitive number	Range: 1~1000
<i>ScreenX1</i>	Vertex 1 x coord.	Range: -1728~2367
<i>ScreenY1</i>	Vertex 1 y coord.	Range: -1936~2159
<i>ScreenX2</i>	Vertex 2 x coord.	Range: -1728~2367
<i>ScreenY2</i>	Vertex 2 y coord.	Range: -1936~2159
<i>ScreenX3</i>	Vertex 3 x coord.	Range: -1728~2367
<i>ScreenY3</i>	Vertex 3 y coord.	Range: -1936~2159
<i>Z</i>	Depth value	Range: 0~16777200

◆Explanation

- Sets the display position of the triangle primitive specified by *TrianglePrimNo*.
- The coordinates of each vertex of the triangle set to the coordinates on the screen (*Screen X1, Screen Y1*), (*Screen X2, Screen Y2*), (*Screen X3, Screen Y3*)
- *Z* specifies the Z-value that indicates the depth of the triangle primitive.

◆Related topics

TRIDISP (see p.59)	TRIOFF (see p.60)
--------------------	-------------------

• Example

TRIPOS 1,0,0,320,224,640,0,0 Set triangle primitive number 1 to have each vertex at (0,0),(320,224),(640,0) and set it to be placed at the very back of the screen.

3D Graphics related

■About 3D space display

◆Model display

The model is placed in a virtual three-dimensional space.
Images taken by a virtual camera are displayed on the screen

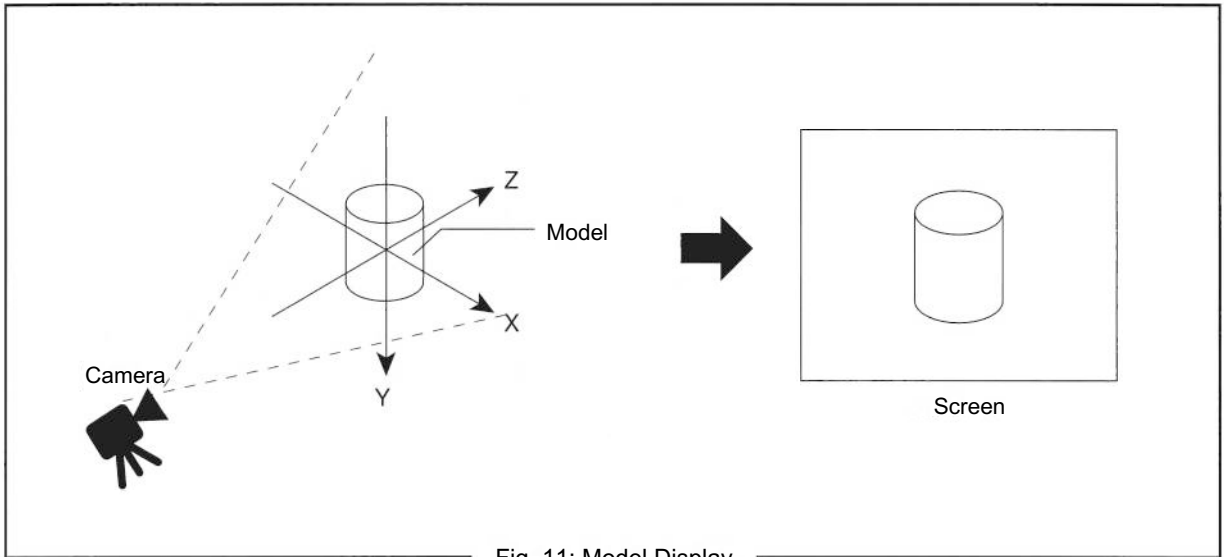


Fig. 11: Model Display

- The position information for the model is specified using the world space coordinate system.

◆World Space

- A virtually set three dimensional space in which models and cameras are located.
- Models in world space have their position represented using world coordinates (x,y,z).
- World space is the absolute space common to all models.

◆Local Space

- A space represented by a coordinate system specific to the model.
- Model rotation is relative to the local coordinate axes.

◆Parent and child models

- A hierarchical relationship can be set between two models. The model that is subordinate to another model is called the child model, and the model that it is subordinate to is called the parent model.
- Translation and rotation of the parent model are reflected in the dependent child models.

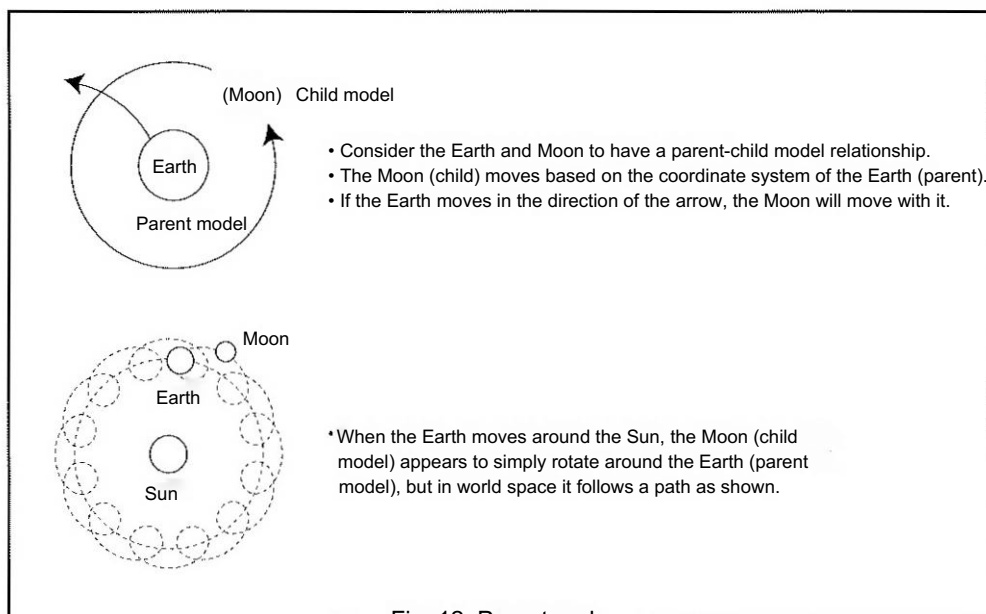


Fig. 12: Parent and child models

■ Models

◆Model Data

- This is a polygon model used in "EZ-BASIC". Model data is created using the modeler "POLY-CRAFT". Depending on the content, texture and palette data may be included.
 - After loading the model data into the model buffer, the model number is linked to the model data in the model buffer.
 - The model number is a management number for handling model data in the program. Numbers can be specified in the range of 1 to 1000, and up to 1000 can be used.
 - When model data is read from a file, it is stored in a model buffer. There are eight model buffers, and each model buffer can store up to ten models.
- Model buffer numbers are designated 1 to 8, and models within a model buffer are designated numbers 1 to 10.

◆Collision detection

- You can use the hitboxes set in the model data to perform collision detection between models. Collision detection involves determining whether or not there is contact, and the distance between contacting models.
- In order to perform collision detection, you must configure the settings for collision detection (hitboxes) in POLY-CRAFT.
- Use the GETCOLL command (see p. 70) or COLLISION command (see p. 101) to detect the collision of a model.

◆Z-sort

- Sort the models by Z-value (see p.44).
- Z sorting is set using the ZSORT command (see p.81).
- This can help mitigate the Z-buffer problem (see p.44).

■Camera

◆Camera Features

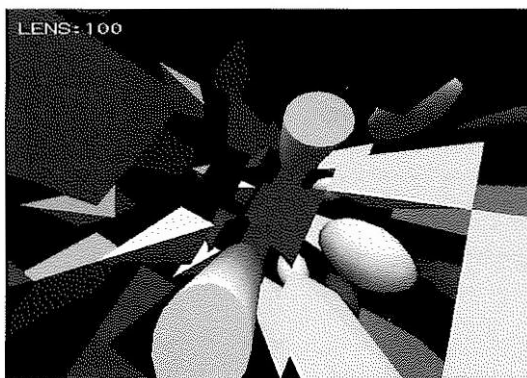
- It is a virtual camera that can capture images in three-dimensional space and display them on-screen.
- Just like a model, a camera is placed in world space and its orientation is specified.
The camera itself is a virtual entity that cannot be seen.
- Up to two cameras can be used.

◆Split Screen

- You can split the screen into left and right or top and bottom displays. You can alternatively display a second screen anywhere on the normal single screen display.
- The image from Camera 1 is projected onto one of the split screens, and the image from Camera 2 onto the other.

◆Lens

- You can set the characteristics of the camera lens. By setting this value, you can obtain wide-angle or telephoto images.



■Light Sources

◆Lights

- You can activate directional lights in world space. Using these, you can give your model the effect of being illuminated by a light from a specific direction.
- You can have up to three lights. Each light is managed by a light number from 1 to 3.
- The light is a directional light source (do not have a position in world space).
- Lights can be set to any color and direction.

◆Ambient light

- Acts as an ambient color.
- Unlike Lights, it has no direction and provide an ambient lighting effect to all models in world space.

■Fog

◆Fog function

- Creates a fog-like effect in three-dimensional space.
- The fog function creates a hazy effect the further away you are from the camera.

◆Fog settings

- The fog has a distance at which the fog begins to appear and a distance at which the fog becomes completely opaque. By setting these two values, you can generate an image in which fog gradually appears between these two points. The distance where fog begins is called the NEAR value, and the distance where it is completely opaque is called the FAR value.
- You can also set the fog color.

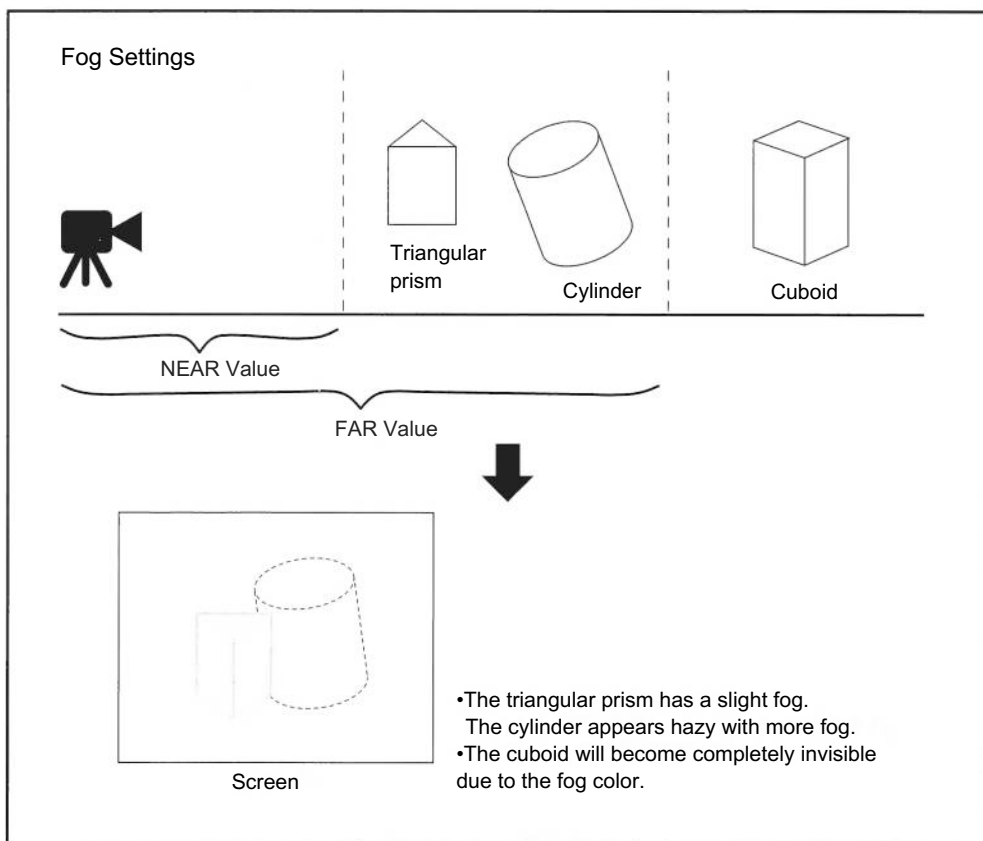


Fig. 13: Fog Settings

■ALTEST

Z value writing using alpha value

◆Format

ALTEST *AlphaValue*

◆Explanation

- Z values will not be written for pixels below the specified alpha value .
- RGB values are not affected.
- Translator note: see <https://www.artdink.co.jp/japanese/title/bs/faq/faqez.html#ez10> for further explanation.

•Example

ALTEST 64 For pixels with an alpha value of 64 or less, no Z value is written.

■AMBCOL

Ambient light setting

◆Format

AMBCOL *Red,Green,Blue*

Item	Content	Note
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255

◆Explanation

- Sets the ambient light in world space to the color specified by *Red*, *Green*, and *Blue*.
- Ambient lighting is applied to objects in world space, but not to text or primitives.

◆Related topics

LIGCOL (see p.74)

•Example

AMBCOL 0,0,255 Makes the ambient light blue.

■ CAMLENS

Sets camera field of view

◆Format

CAMLENS *CameraNo.Lens*

Item	Content	Note
<i>CameraNo</i>	Camera number	Range: 1~2
<i>Lens</i>	Lens value	Range: 100~10000 Default:512

◆Explanation

- Sets the angle of view of the camera specified by *CameraNo*.
- Specify a value for *Lens* that corresponds to the angle of view.
Increasing this value will create a telephoto lens effect, while decreasing it will create a wide-angle lens effect.

•Example

CAMLENS 1,2000 Sets camera number 1 to a telephoto lens effect.

■ CAMPOS

Set camera location

◆Format

CAMPOS *CameraNo, WorldX, WorldY, WorldZ*

Item	Content	Note
<i>CameraNo</i>	Camera number	Range: 1~2
<i>WorldX</i>	World x coordinate	
<i>World Y</i>	World y coordinate	
<i>WorldZ</i>	World z coordinate	

◆Explanation

- Sets the position of the camera specified by *CameraNo*.
- Positions the camera at world coordinates (*WorldX, WorldY, WorldZ*).
- The initial position of camera 1 is world coordinates (0,0,-100), and the initial position of camera 2 is world coordinates (0,0,100).

◆Related topics

CAMVEC (see p.68) CAMROT (see p.67)

•Example

CAMPOS 1,0,0,0 Sets camera number 1 to the origin (0,0,0) in world space.

◆Format

CAMROT CameraNo, RotX, RotY, RotZ

Item	Content	Note
CameraNo	Camera number	Range: 1~2
RotX	X-axis angle	Units: Radians
RotY	Y-axis angle	Units: Radians
RotZ	Z-axis angle	Units: Radians

◆Explanation

- The direction of the camera specified by *CameraNo* is set by the angle of each axis.
- Specify the angle of each axis in *RotX*, *RotY*, and *RotZ*.

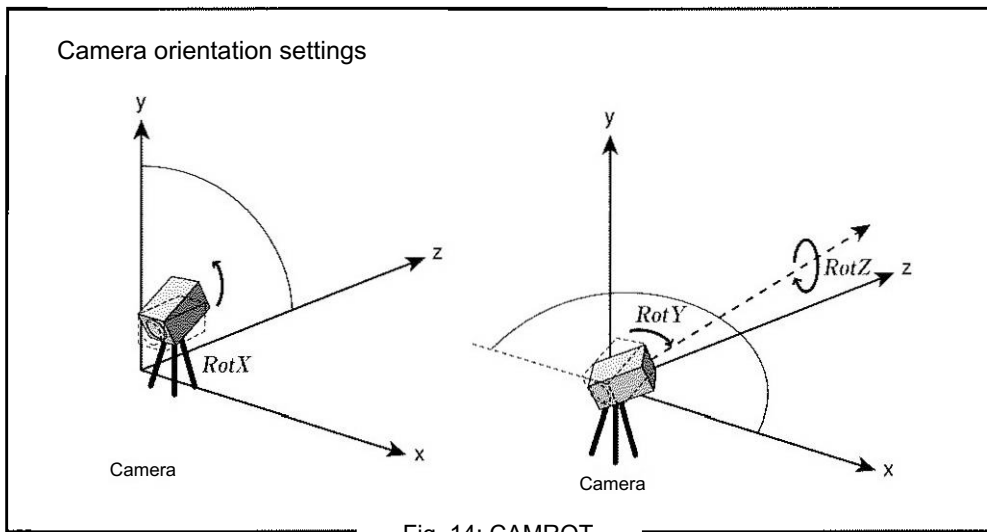


Fig. 14: CAMROT

- The units used are radians. To convert an angle to radians, use the RAD function (see p.98).

◆Related topics

CAMPOS (see p.66) CAMVEC (see p.68)

• Example 1

CAMROT 1,PI/2,0,0
CAMROT 1,0,0,RAD(90)

Points camera number 1 straight down.
Tilt camera number 1 90 degrees to the left.

• Example 2

```
100 MODDISP 1
110 DO
120 X=X+RAD(PAD(1,3)-PAD(1,4))/80
130 Y=Y+RAD(PAD(1,2)-PAD(1,1))/80
140 Z=Z+RAD(PAD(1,10)-PAD(1,9))/80
150 CAMROT 1,X,Y,Z
160 PRINT IP(DEG(X)), IP(DEG(Y)), IP(DEG(Z))
170 FSYNC 1
180 LOOP
190 END
```

Use the up and down buttons to rotate the camera on the X axis.
Use the left and right buttons to rotate the camera on the Y axis.
Use the L1 and R1 buttons to rotate the camera on the Z axis.

◆Format

CAMVEC *CameraNo*, *VectorX*, *VectorY*, *VectorZ*

Item	Content	Note
<i>CameraNo</i>	Camera number	Range: 1~2
<i>VectorX</i>	Vector X-component	
<i>VectorY</i>	Vector y-component	
<i>VectorZ</i>	Vector z-component	

◆Explanation

- Sets the direction of the camera specified by *CameraNo* using a vector.
- *VectorX*, *VectorY*, *VectorZ* are used to specify the direction vector.

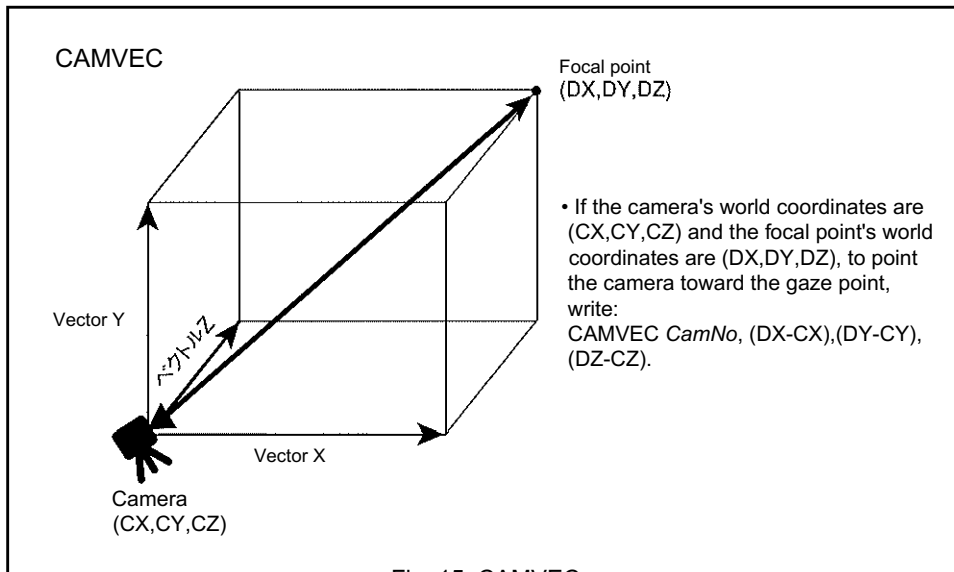


Fig. 15: CAMVEC

◆Related topics

CAMPOS (see p.66)

CAMROT (see p.67)

•Example

```

100 CX,CY=0
110 CZ=100
120 MODDISP 1
130 MODDISP 2
140 DO
150 S=S+1
160 DX=100*SIN(RAD(S))
170 DY,DZ=0
180 MODPOS 1,DX,DY,DZ
190 CAMPOS 1,CX,CY,CZ
200 CAMVEC DX-CX,DY-CY,DZ-CZ
210 FSYNC 1
220 LOOP
230 END

```

Variables for camera 1's world coordinates

Variables for model 1's world coordinates

Position model 1

Position model 2

Camera 1 points at model 1

◆Format

FOGCOL *Red,Green,Blue*

Item	Content	Note
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255

◆Explanation

- Sets the color of the world space fog.
- *Red*, *Green*, and *Blue* values determine intensity of each color component.
- Fog affects models only. Primitives and text are unaffected.

◆Related topics

FOGSET (see p.69)

• Example

FOGCOL 255,255,255 Sets fog color to white.

FOGSET

Set fog effect range

◆Format

FOGSET *Near,Far*

Item	Content	Note
<i>Near</i>	Fog start distance	Range: 0~10000000
<i>Far</i>	Fog complete distance	Range: 0~10000000

◆Explanation

- Sets range of the fog effect.
- Effect range is determined by distance from the camera.
Near determines when the fog effect begins, *Far* determines distance when fog completely obscures view.

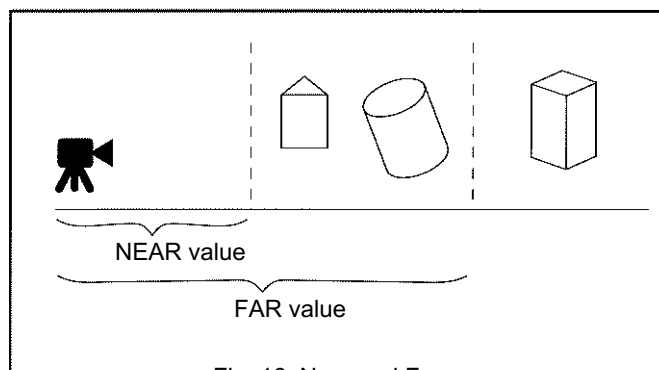


Fig. 16: Near and Far values

- If $Near \geq Far$, an error is thrown.
- Fog affects models only. Primitives and text are unaffected.

◆Related topics

FOGCOL (see p.69)

• Example

FOGSET 20,100 In world space, the fog effect starts a distance of 20 from the camera, and positions a distance greater than 100 away are colored completely the color of the fog.

◆Format

GETCOLL *VarModel, VarX, VarY, VarZ, ModelSrc, ModelStart, ModelEnd*

Item	Content	Note
<i>VarModel</i>	Output variable (Model number)	
<i>VarX</i>	Output variable (X vector)	
<i>VarY</i>	Output variable (Y vector)	
<i>VarZ</i>	Output variable (Z vector)	
<i>ModelSrc</i>	Model number for collision target	
<i>ModelStart</i>	Start model number for collision check	
<i>ModelEnd</i>	End model number for collision check	

◆Explanation

- For the model specified by *ModelSrc*, a collision check is performed with the models numbered *ModelStart* to *ModelEnd*, and the distance between the colliding models is returned as a vector.
If there is a contacting model, the vector between that model and the *ModelSrc* model is assigned to each output variable.
- The assigned vector is calculated from the vector from the center of the collision detection area of each model.
- If there is no colliding model, *VarModel* is set to 0.
- Collision detection settings must be configured within the model data.

◆Related topics

COLLISION (see p.101)

• Example

GETCOLL N,X,Y,Z,1,10,20 Perform collision detection for models 10 to 20 against model 1, store the colliding model number in variable N.
The vectors from the center of the collision area of model 1 to the center of the collision area of the colliding model are obtained in variables X, Y, and Z.

◆Format

GETLPOS *LocalX,LocalY,LocalZ,ModelNo,WorldX,WorldY,WorldZ*

Item	Content	Note
<i>LocalX</i>	Output variable Local x coord.	
<i>LocalY</i>	Output variable Local y coord.	
<i>LocalZ</i>	Output variable Local z coord.	
<i>ModelNo</i>	Model number	Range: 1~1000
<i>WorldX</i>	World x coordinate	
<i>WorldY</i>	World y coordinate	
<i>WorldZ</i>	World z coordinate	

◆Explanation

- Returns the local coordinates of (*WorldX, WorldY, WorldZ*) with respect to the model specified by *ModelNo*.

◆Related topics

GETWPOS (see p.73) GETSPOS (see p.72)

•Example

GETLPOS A,B,C,5,100,50,20 Writes the local coordinate values of model 5 at world coordinates (100,50,20) into variables A, B, and C.

◆Format

GETSPOS *ScreenX,ScreenY,ScreenZ, CameraNo, WorldX,WorldY,WorldZ*

Item	Content	Note
<i>ScreenX</i>	Output variable Screen x coord.	
<i>ScreenY</i>	Output variable Screen y coord.	
<i>ScreenZ</i>	Output variable Screen z coord.	
<i>CameraNo</i>	Camera number	Range:1~2
<i>WorldX</i>	World x coordinate	
<i>WorldY</i>	World y coordinate	
<i>WorldZ</i>	World z coordinate	

◆Explanation

- Calculates the screen coordinates position specified by world coordinates (*WorldX, WorldY, WorldZ*).
- The camera is specified with *CameraNo*.
- The screen coordinates are assigned to the output variables (*ScreenX, ScreenY, ScreenZ*).

◆Related topics

GETLPOS (see p.71) GETWPOS (see p.73)

• Example 1

GETSPOS SX,SY,SZ,1,0,0,0 The screen coordinates of world coordinates (0,0,0) as seen from camera number 1 are written into variables (SX, SY, SZ).

• Example 2

```

100 TEXTMODE 1
110 MODDISP 1
120 DO
130  X=(PAD(1,14)-128)/8      Left stick changes the model's world X coordinate value
140  Y=(PAD(1,13)-128)/8      Left stick changes the model's world Y coordinate value
150  Z=PAD(1,3)               Directional pad changes the model's Z coordinate value
160  GETSPOS SX,SY,SZ,1,X,Y,Z  Get screen coordinates from world space coordinates and camera number
170  MODPOS 1,X,Y,Z
180  CLEAR
190  LOCATE SX/23,SY/10        Specify the text drawing position based on the acquired screen coordinate values
200  PRINT INT(SX),INT(SY),INT(SZ) Displays the current screen coordinate value with only the integer part displayed.
210  FSYNC 1
220 LOOP
230 END

```

◆Format

GETWPOS *WorldX,World Y,WorldZ,ModelNo,LocalX,LocalY,LocalZ*

Item	Content	Notes
<i>WorldX</i>	Output variable World x coord.	
<i>WorldY</i>	Output variable World y coord.	
<i>WorldZ</i>	Output variable World z coord.	
<i>ModelNo</i>	Model number	Range: 1~1000
<i>LocalX</i>	Local x coordinate	
<i>LocalY</i>	Local y coordinate	
<i>LocalZ</i>	Local z coordinate	

◆Explanation

- Obtains the world coordinates of the local coordinates (*LocalX*, *LocalY*, *LocalZ*) of the model specified by *ModelNo*.
- The world coordinates are assigned to the output variables (*WorldX*, *WorldY*, *WorldZ*).

◆Related topics

GETLPOS (see p.71) GETSPOS (see p.72)

• Example

```

100 MODDISP 1
110 MODDISP 2
120 DO
130  A=A+(PAD(1,1)-PAD(1,2))/100      Model 1 is controlled by using the left and right directional buttons.
140  MODPOS 1,A,0,100
150  IF PAD(1,8)>0 THEN                While square is pressed, line 160 is executed.
160    GETWPOS X,Y,Z,1,0,20,0          Find the world coordinates of the local coordinates (0,20,0) of model 1.
170  END IF
180  MODPOS 2,X,Y,Z                    Place the second model at world coordinates (X,Y,Z) .
190  FSYNC 1
200 LOOP
210 END

```

◆Format

LIGCOL *LightNo,Red,Green,Blue*

Item	Content	Note
<i>LightNo</i>	Light number	Range: 1~3
<i>Red</i>	Red intensity	Range: 0~255
<i>Green</i>	Green intensity	Range: 0~255
<i>Blue</i>	Blue intensity	Range: 0~255

◆Explanation

- Sets the color of the light specified by *LightNo*.
Lights illuminate models in world space.
There are three directional light sources, each with its own color.
- Specify the values of each color component in *Red*, *Green*, and *Blue*.
- Lights only affect models, not primitives or text.

◆Related topics

LIGVEC (see p.75)

◆Example/ Changing light color by controller input

```

100 MODDISP 1
110 DO
120  R=PAD(1,8)      Set the red component by pressure input from the square button
130  G=PAD(1,5)      Set the green component by pressure input from the triangle button
140  B=PAD(1,6)      Set the blue component by pressure input from the circle button
150  LIGCOL 1,R,G,B  Changes the light color to the values of variables R, G, and B.
160  PRINT R,G,B
170 LOOP
180 END

```

◆Format

LIGVEC *LightNo*, *VectorX*, *VectorY*, *VectorZ*

Item	Content	Note
<i>LightNo</i>	Light number	Range: 1~3
<i>VectorX</i>	X vector	
<i>VectorY</i>	Y vector	
<i>VectorZ</i>	Z vector	

◆Explanation

- Sets the direction of the light specified by *LightNo*.
- *VectorX*, *VectorY*, and *VectorZ* specify the direction the light shines.

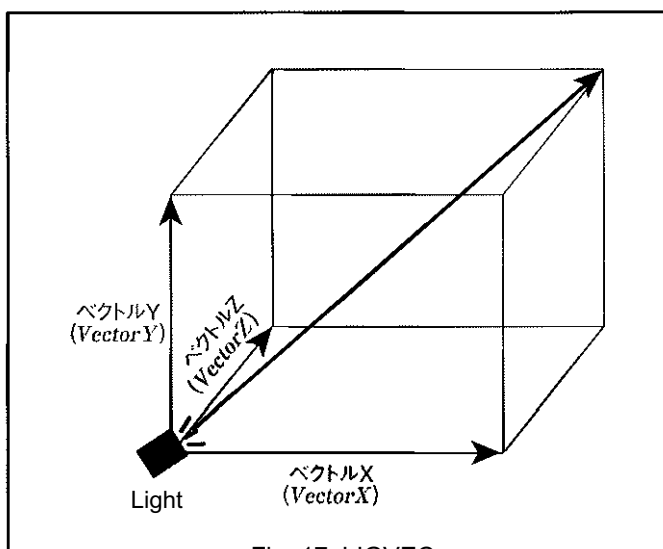


Fig. 17: LIGVEC

◆Related topics

LIGCOL (see p.74)

•Example

```

100 M0DDISP 1
110 DO
120  X=PAD(1,8)-128
130  Y=PAD(1,5)-128
140  Z=PAD(1,6)-128
150  LIGVEC 1,X,Y,Z
160  PRINT X,Y,Z
170 LOOP
180 END

```

Set the x vector by pressure input from the square button
Set the y vector by pressure input from the triangle button
Set the z vector by pressure input from the circle button

The direction of the light is set by the values of X, Y, and Z.

◆Format

MODDISP *ModelNo*

Item	Content	Note
<i>ModelNo</i>	Model number	Range: 1~1000

◆Explanation

- Displays model specified by *ModelNo*.

◆Related topics

MODJOIN (see p.77)	MODOFF (see p.78)	MODOPT (see p.79)
MODPOS (see p.79)	MODSET (see p.81)	MODROT (see p.80)

•Example/ Model display example

```

100 FOR A=1 TO 10
110  MODDISP A
120  MODPOS A, (A-5)*10,0,0
130 NEXT A
140 END
    
```

Use a loop to display the models all at once.
Display the model indicated by the value of variable A.

◆Format

MODJOIN *ModelNo*,*ParentModelNo*

Item	Content	Note
<i>ModelNo</i>	Model number	Range:1~1000
<i>ParentModelNo</i>	Parent model number	Range:0~1000

◆Explanation

- The model specified by *ModelNo* will be linked to the model specified by *ParentModelNo*.
- *ParentModelNo* and *ModelNo* are in a parent-child relationship, and the child model *ModelNo* uses the local coordinate system of the parent model as its reference.
- The specified values must be such that *ParentModelNo*<*ModelNo*.
- If you specify 0 for *ParentModelNo*, the child model will be unlinked.

◆Related topics

MODDISP (see p.76)	MODOFF (see p.78)	MODOPT (see p.79)
MODPOS (see p.79)	MODSET (see p.81)	MODROT (see p.80)

•Example/ Connecting/disconnecting a model to a parent model

```

100 MODDISP 1          Parent model
110 MODDISP 2          Child model
120 DO
130  X=(PAD(1,14)-128)/6
140  Y=(PAD(1,13)-128)/6
150  IF=PAD(1,8)>0 THEN MODJOIN 2,1  Pressing the square button connects it to the parent model.
160  IF=PAD(1,7)>0 THEN MODJOIN 2,0  Pressing the x button disconnects it from the parent model.
170  MODPOS 1,X,Y,0      Move the parent model using the controller
180  R=R+1              Updating the rotation angle of the child model
190  MODROT 2,0,RAD(R),0
200  MODPOS 2,10,0,0
210  FSYNC 1
220  LOOP
230  END

```

When linked to the parent, the MODPOS command is executed in the parent's local coordinate system, and when unlinked, it is executed in the world coordinate system.

Explanation

- 150 Model number 1 and model number 2 are in a parent-child relationship.
This means that the model placement commands in lines 190-200 will be based on the local coordinates of the parent model.
If you move the parent model, the child model will also move.
- 160 Delete the parent-child relationship.
This means that all commands in lines 190 to 200 will be based on the world coordinate system.
Even if the parent model, model number 1, moves, the child model, model number 2, is not affected.

◆Format

MODLOAD [*DirectoryName* /] *FileName,ModelBufferNo*

Item.....	Note
<i>ModelBufferNo</i>	Range: 1~8

◆Explanation

- MOD format model data is read from the file specified by *FileName* and stored in the model buffer specified by *ModelBufferNo*.
- If you specify *DirectoryName*, the file in that directory will be read.
If you omit *DirectoryName*, the file will be read from the "SYSTEM" directory.
- There are eight model buffers. Each model buffer can store up to ten different models.

◆Related topics

MODSET (see p.81) MODDISP (see p.76)

• Example

MODLOAD "TEST",1	Load the data in the file "TEST.MOD" into model buffer number 1.
MODLOAD FNAM\$,3	The file with name specified by the string variable FNAM\$ is read into model buffer number 3.

◆Format

MODOFF *ModelNo*

Item.....	Content	Note
<i>ModelNo</i>	Model number	Range: 1~1000

◆Explanation

- Hides the model specifeid by *ModelNo*.

◆Related topics

MODDISP (see p.76)

•Example

MODOFF 123	Hides model number 123.
------------	-------------------------

◆Format

MODOPT *ModelNo,Scale,Light,Fog,Alpha*

Item	Content	Note
<i>ModelNo</i>	Model number	Range:1~1000
<i>Scale</i>	Scale factor	Range:0.001~1000 Default:1
<i>Light</i>	Light calculation	Range:0:No light applied 1:Light applied
<i>Fog</i>	Fog effect	Range:0:No fog applied 1:Fog applied
<i>Alpha</i>	Alpha mode	Mode:0:Subtract 1:Normal 2:Add mode

◆Explanation

- Set the options for the model specified by *ModelNo* .
- Sets the model's scale factor, whether to calculate lighting, whether to apply fog, and the alpha mode.

• Example

MODOPT 1,2,0,1,1 Make model number 1 twice as large and apply no lights.

◆Format

MODPOS *ModelNo, WorldX, WorldY, WorldZ*

Item	Content	Note
<i>ModelNo</i>	Model number	Range: 1~1000
<i>WorldX</i>	World x coordinate	
<i>World Y</i>	World y coordinate	
<i>WorldZ</i>	World z coordinate	

◆Explanation

- Set the world coordinates where the model specified by *ModelNo* will be placed.
- Specify the world coordinates in the values (*WorldX, WorldY, WorldZ*).

◆Related topics

MODDISP (see p.76) MODJOIN (see p.77) MODOPT (see p.79)
MODOFF (see p.78) MODROT (see p.80) MODSET(see p.81)

•Example/ Moving a model via MODPOS

```

100 MODDISP 1
110 DO
120  X=X+(PAD(1,1)-PAD(1,2))/80      Update the value of variable X with controller input
130  Y=Y+(PAD(1,4)-PAO(1,3))/80      Update the value of variable Y with controller input
140  Z=Z+(PAD(1,11)-PAD(1,9))/80     Update the value of variable Z with controller input
150  MODPOS 1,X,Y,Z                 Place at world coordinate values X, Y, Z
160  FSYNC 1
170 LOOP
180 END

```


Format

MODROT ModelNo, RotX, RotY, RotZ

Item	Content	Note
ModelNo	Model number	Range: 1~1000
RotX	X axis angle	Units: Radians
RotY	Y axis angle	Units: Radians
RotZ	Z axis angle	Units: Radians

Explanation

- Sets the angle of the model specified by ModelNo .
- Set the angle for each axis using the angles (RotX, RotY, RotZ).
- Angles are measured in radians. To convert angles to radians, use the RAD function (see p.98).

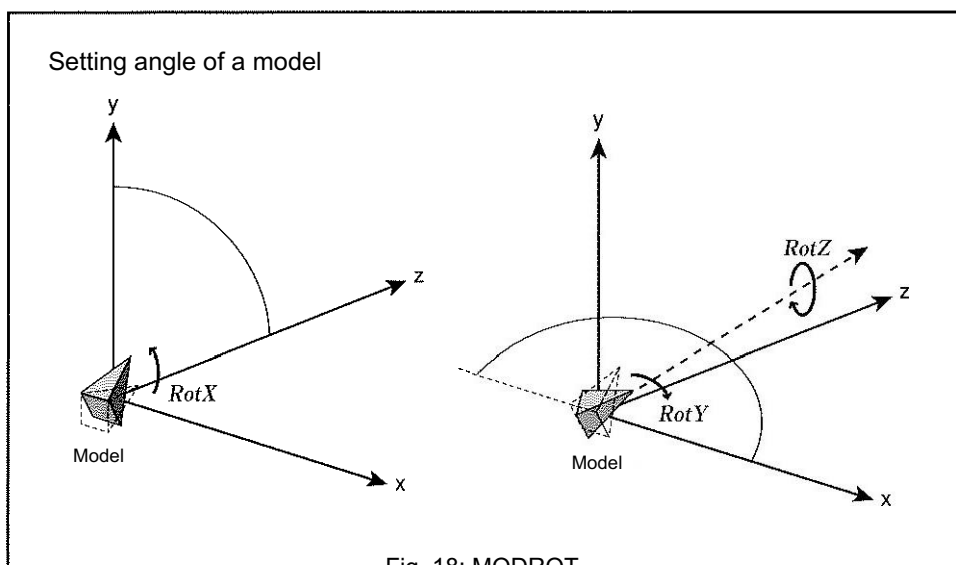


Fig. 18: MODROT

Related topics

MODJOIN (see p.77)

MODPOS (see p.79)

Example 1

MODROT 1,PI/2,0,0 Rotate model 1 about the x-axis by pi/2 radians.
 MODROT 1,RAD(90),RAD(-30),RAD(45) Rotate model 1 about the x-axis by 90 degrees, about the y axis by -90 degrees, and about the z axis by 45 degrees.

Example 2

```

100 MODDISP 1
110 DO
120  X=X+(PAD(1,4)-PAD(1,3))/80      Sets the X-axis tilt via controller input.
130  Y=Y+(PAD(1,2)-PAD(1,1))/80      Sets the Y-axis tilt via controller input.
140  Z=Z+(PAD(1,10)-PAD(1,9))/80      Sets the Z-axis tilt via controller input.
150  MODROT 1,RAD(X),RAD(Y),RAD(Z) Converts the angles X, Y, and Z to radians and executes MODROT.
160  PRINT INT(X),INT(Y),INT(Z)      Displays the integer part of the angle X, Y, and Z values.
170  FSYNC 1
180 LOOP
190 END
  
```

■MODSET

Register model to model number

◆Format

MODSET *ModelNo,ModelBuffNo,ObjNo*

Item	Content	Note
<i>ModelNo</i>	Model number	Range: 1~1000
<i>ModelBuffNo</i>	Model buffer number	Range: 1~8
<i>ObjNo</i>	Object number in model buffer	Range: 1~10

◆Explanation

- Associates a model number with a model stored in the model buffer.
- The model data number *ObjNo* in the model buffer specified by *ModelBuffNo* is set to model *ModelNo*.
- The same model data can be used for multiple model numbers and treated as separate characters.
- If you specify model data that has not been loaded into the model buffer using the MODLOAD command (see p.78), an error will occur.

◆Related topics

MODLOAD (see p.78) MODJOIN (see p.77) MODOPT (see p.79)

•Example

MODSET 3,1,1 Assign the first object in model buffer number 1 to model number 3.

■ZSORT

Z-sort models

◆Format

ZSORT *StartModelNo, EndModelNo*

Item	Content	Note
<i>StartModelNo</i>	First model number to sort	Range: 1~1000
<i>EndModelNo</i>	Last model number to sort	Range: 1~1000

◆Explanation

- When drawing objects, sort models by their Z value from the screen and draw them in order of greatest depth first.
- Once you set it up, the actual Z sorting will be done automatically when drawing.
- If you specify the same number for *StartModelNo* and *EndModelNo*, the Z sorting range setting will be canceled and Z sorting will not be performed. In this case, the number you specify can be any number.
- If you do not use semi-transparent objects, there is no need to set this.
- This function can reduce the Z- buffer problem (see p.44).
- If Z-sorting is not set, models will be drawn in order of model number.

◆Related topics

ORDER (see p.38)

•Example

ZSORT 100,200 Models numbered 100 to 200 are z-sorted and drawn in order starting with furthest.

.....

Sound

.....

■BGMLOAD

Load BGM

◆Format

BGMLOAD [*DirectoryName* /] *FileName*, *BGMBufferNo*

Item	Note
<i>BGMBufferNo</i>	Range:1~4

◆Explanation

- BGM data is read from the specified file name and stored in the BGM buffer specified by the BGM buffer number.
- If you specify a directory name, the file will be read from that directory. If you omit the directory name, the file will be read from the "SYSTEM" directory.

◆Related Topics

BGMPLAY (see p.82) SELOAD (see p.83)

• Example

BGMLOAD "SOUND/BGM1",1 BGM1 in the SOUND directory is read and the BGM data is stored in BGM buffer number 1.

■BGMPLAY

Play BGM

◆Format

BGMPLAY *BgmNo*

Item	Content	Note
<i>BgmNo</i>	BGM buffer number	Range: 1~4

◆Explanation

- Plays the BGM data stored in the BGM buffer specified by *BgmNo*.

◆Related topics

BGMLOAD (see p.82) SEPLAY (see p.84) BGMSTOP (see p.83)

• Example

BGMPLAY 3 Plays the BGM data in BGM buffer 3.

■BGMSTOP

Stops BGM

◆Format

BGMSTOP

◆Explanation

- Stops BGM playback.

◆Related topics

BGMPLAY (see p.82)

■SELOAD

Load sound effects

◆Format

SELOAD [*DirectoryName* /] *FileName*

◆Explanation

- Reads SE data from the file specified in by *FileName* .
If you specify a directory name, the file will be read from that directory.
If the directory name is omitted, the file is read from the "SYSTEM"

◆Related topics

SEPLAY (see p.84)

• Example

SELOAD "SE/SE01"

Loads the SE file SE01 in the "SE" directory.

◆Format

SEPLAY *SeNo,LeftVol,RightVol*

Item	Content	Note
<i>SeNo</i>	Sound effect number	Range: 1~128
<i>LeftVol</i>	Left volume	Range: 0~255
<i>RightVol</i>	Right volume	Range: 0~255

◆Explanation

- Plays the sound effect specified by *SeNo*.
- Sound effects will be played from the left and right speakers at the volume specified by *LeftVol* and *RightVol*.

◆Related topics

SELOAD (see p.84)

• Example/ Sound effect test program

```

100 DO WHILE PAD (1,19)=0
110  CLEAR
120  SE=SE+(PAD(1,1)-PAD(1,2))*0.001
130  V=MAX(-255,MIN(255,V+(PAD(1,9)-PAD(1,10))*0.005))
140  L=MIN(255,255+V)
150  R=MIN(255,255-V)
160  IF PAD(1,6)>0 THEN
170    IF FLGB=0 THEN
180      SEPLAY SE,L,R
190      FLGB=1
200    END IF
210  ELSE
220    FLGB=0
230  END IF
240  SE=MAX(1,MIN(100,SE))
250  PRINT "SE:";INT(SE),"L:",INT(L),"R:",INT(R)
260  FSYNC 1
270 LOOP

```

<Explanation>

- 120 The directional buttons left and right change the sound effect number to be played.
 130 Use the L1 and R1 buttons on the controller to adjust the left and right volume.
 140 Left volume setting
 150 Right Volume Setting
 160 Press the O button to play a sound effect.

◆Format

VOLUME *Volume*

Item	Content	Note
<i>Volume</i>	Volume	Range: 0~255

◆Explanation

- Sets the sound system's volume to the value specified by *Volume*.

◆Related topics

BGMPLAY (see p.82) SEPLAY (see p.84)

◆Example/ Program for manipulating volume

```

100 BGMPLAY 1
110 TRIDISP 1
120 DO UNTIL PAD(1,19)=1
130  CLEAR
140  V=MAX(1,MIN(255,V+(PAD(1,3)-PAD(1,4))*0.005))
150  VOLUME V
160  TRIC0L3 1,255,V,V,128,V,255,0,128,0,255,V,128
170  TRPOS 1,320,220,320-V,220-V*0.8,320+V,220-V*0.8,0
180  PRINT "VOL:";INT(V)
190  FSYNC 1
200 LOOP
210 INIT

```

<Explanation>

100 Play BGM data.
140 Up and down directional pad sets the volume in the range of 1 to 255.
150 Sets the volume to the value of variable V.

String functions

■CHR\$

Get the character corresponding to JIS code number

◆Format

CHR\$(x)

Item	Content	Note
x	JIS code number	0~255

◆Explanation

- Returns character corresponding to JIS code.

Table 1: JIS Code Table

code	Key	code	Key	code	Key	code	Key	code	Key
0	・	63	?	97	a	163	」	197	ナ
・	・	64	@	98	b	164	、	198	ニ
・	・	65	A	99	c	165	・	199	ヌ
・	・	66	B	100	d	166	ヲ	200	ネ
33	!	67	C	101	e	167	ア	201	ノ
34	"	68	D	102	f	168	イ	202	ハ
35	#	69	E	103	g	169	ウ	203	ヒ
36	\$	70	F	104	h	170	エ	204	フ
37	%	71	G	105	i	171	オ	205	ヘ
38	&	72	H	106	j	172	ヤ	206	ホ
39	'	73	I	107	k	173	ユ	207	マ
40	(	74	J	108	l	174	ヨ	208	ミ
41	)	75	K	109	m	175	ツ	209	ム
42	*	76	L	110	n	176	ー	210	メ
43	+	77	M	111	o	177	ァ	211	モ
44	,	78	N	112	p	178	ィ	212	ヤ
45	-	79	O	113	q	179	ゥ	213	ユ
46	.	80	P	114	r	180	ヱ	214	ヨ
47	/	81	Q	115	s	181	ォ	215	ラ
48	0	82	R	116	t	182	カ	216	リ
49	1	83	S	117	u	183	キ	217	ル
50	2	84	T	118	v	184	ク	218	レ
51	3	85	U	119	w	185	ケ	219	ロ
52	4	86	V	120	x	186	コ	220	ワ
53	5	87	W	121	y	187	サ	221	ン
54	6	88	X	122	z	188	シ	222	''
55	7	89	Y	123	{	189	ス	223	・
56	8	90	Z	124		190	セ	・	・
57	9	91	[	125	}	191	ソ	・	・
58	:	92	¥	126	~	192	タ	・	・
59	;	93	]	・	・	193	チ	・	・
60	<	94	^	・	・	194	ツ	・	・
61	=	95	_	161	。o	195	テ	・	・
62	>	96	`	162	「	196	ト	255	・

◆Related topics

- ORD(see p.88)

DATE\$

Get date as string

◆Format

DATE\$

◆Explanation

- Returns the current date as a string.
- The date string is obtained in the form of a four-digit year, two-digit month, and two-digit day.

◆Related topics

TIME (see p.100)

TIME\$ (see p.90)

LCASE\$

Replace uppercase letters with lowercase letters

◆Format

LCASE\$(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Replaces all uppercase letters in the value of a\$ with lowercase letters and returns the resulting value.
- Characters other than half-width uppercase English letters will be ignored.

◆Related topics

UCASE\$ (see p.91)

Example

LCASE\$ ("Basic Studio 1.0") Returns string "basic studio 1.0"

LEN

Get string length

◆Format

LEN(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Evaluates the length of the value of a\$ and returns that value.
- Each full-width character is evaluated as having a length of 2.
- Spaces are also considered to be one character long.

•Example

PRINT LEN("BASICプログラム") Displays the length of the string given as an argument.
Full-width characters have a length of 2 per character.

<Execution result>

15

■LTRIM\$

Delete leading spaces

◆Format

LTRIM\$(a\$)

Item	Content	Format
a\$	String expression	

◆Explanation

- Evaluates the value of a\$ and returns the string obtained by removing all spaces that precede non-blank characters.

◆Related topics

RTRIM\$ (see p.90)

•Example

PRINT LTRIM\$(" A B cd") Prints the passed argument after removing leading spaces.

<Execution result>

A B cd

■ORD

Get JIS code of string

◆Format

ORD(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Evaluates a\$ and returns the corresponding JIS code number.

◆Related topics

CHR\$ (see p.86)

•Example

PRINT ORD("#") Prints the JIS code for the character "#".

■ POS

Get the first character position of the matched string

◆ Format

POS(*a\$*,*b\$*)

Item	Content	Note
<i>a\$</i>	String expression	
<i>b\$</i>	String expression	

◆ Explanation

- Returns the position of the first character in the value of *a\$* that first matches the value of *b\$*.
- If there is no match, returns 0.
- If *b\$* is the empty string "", it will always be 1 regardless of the value of *a\$*.

• Example

POS("ABCDEFGH", "CD")	Matches the third character "C" and returns 3.
POS("ABCDEFGH", "XYZ")	Returns 0 since there is no match.
PRINT POS("BASIC", "")	Since an empty string is specified, it returns 1.

■ REPEAT\$

Returns repeatedly concatenated string

◆ Format

REPEAT\$(*a\$*, *x*)

Item	Content	Note
<i>a\$</i>	String expression to repeat	
<i>x</i>	Repeat count	Range: above 0

◆ Explanation

- Concatenates the value of *a\$* *x* times and returns the resulting string.
- *x* must be greater than or equal to 0. If 0 is specified, an empty string is returned.
- If the length of the returned string is greater than 512, an error occurs.

◆ Example

A\$=REPEAT\$("ABC", 5)	The string "ABC" repeated five times is assigned to the string variable A\$.
------------------------	--

■ RTRIM\$

Delete trailing spaces

◆Format

RTRIM\$(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Returns the value of a\$ with all trailing spaces removed.

◆Related topics

LTRIM\$ (see p.88)

• Example

RTRIM\$(" A B C ") Removes trailing spaces and returns " A B C".

■ STR\$

Get string from numerical value

◆Format

STR\$(x)

Item	Content	Note
x	Numerical expression	

◆Explanation

- Evaluates x and returns its value as a string.

◆Related topics

VAL (see p.91)

• Example

A\$=STR\$(5/2) Evaluates 5/2 and assigns the resulting value 2.5 as a string to the string variable A\$.
B\$=STR\$(-3E2) Evaluates -3E2 and assigns the resulting value -300 as a string to the string variable B\$.

■ TIME\$

Get current time as string

◆Format

TIME\$

◆Explanation

- Calculates the current time and returns it as a string.
- Time is displayed using a 24-hour clock, with hours, minutes, and seconds separated by colons (:).
- The value returned by the TIME\$ function at midnight is " 00:00:00".

◆Related topics

TIME (see p.100) DATE\$ (see p.87)

• Example

A\$=TIME\$ Assign the current time to string A\$

◆Format

UCASE\$(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Replace all lowercase letters in the evaluation value of a\$ with uppercase letters and return the resulting value.
- Characters other than lowercase English letters will be ignored.

◆Related topics

LCASE\$ (see p.87)

• Example

UCASE\$("Basic Studio 1.0") Returns string "BASIC STUDIO 1.0"

◆Format

VAL(a\$)

Item	Content	Note
a\$	String expression	

◆Explanation

- Returns the evaluation of a\$ as a number.
- a\$ can contain half-width digits 0 to 9, plus and minus signs, decimal points, and exponent symbols.
- If a\$ is preceded by a string other than a character that can be evaluated as a number, an error occurs.
- If a\$ contains multiple strings that can be evaluated as numbers, only the first value is returned.
- Spaces before and after a\$ are ignored.

◆Related topics

STR\$ (see p.90)

• Example

VAL("-12.34")	Returns the number -12.34.
VAL("300"&" .400")	Returns the number 300.4.
VAL("1E3")	The exponential expression 1E3 is evaluated to obtain the number 1000.
VAL("34P0INTS")	Returns the number 34.
VAL("100 OR 200")	Returns the number 100.
VAL(" 2E-3 ")	Returns the number 0.002.

Numeric Functions

■ABS

Obtain absolute value

◆Format

ABS(*x*)

Item	Content	Note
<i>X</i>	Numeric expression	

◆Explanation

- Evaluates *x* and returns its absolute value.

■ACOS

Inverse cosine

◆Format

ACOS(*x*)

Item	Content	Note
<i>X</i>	Numeric value	Units: Radians

◆Explanation

- Evaluates *x* and returns its arc cosine.
- Return value range: $0 \leq \text{ACOS}(x) \leq \pi$

■ANGLE

Get angle from 2D vector

◆Format

ANGLE(*x*,*y*)

Item	Content	Note
<i>x</i>	X-vector	
<i>y</i>	Y-vector	

◆Explanation

- Returns the angle of the vector connecting the origin and the coordinates (*x*, *y*) relative to the X axis.
- The returned value is in radians and is in the range $-\pi \leq \text{ANGLE}(x,y) \leq \pi$.
- Counterclockwise is positive.
- If both the arguments *x* and *y* are set to 0, an error occurs.

•Example

ANGLE(1,1) The value 0.785398(=45°) is obtained

■ASIN

Inverse sine

◆Format

ASIN(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates x and returns its arc sine.
- The returned value is in radians and is in the range $-\pi/2 \leq \text{ASIN}(x) \leq \pi/2$.

■ATN

Inverse tangent

◆Format

ATN(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates x and returns its arctangent.
- The returned value is in radians and is in the range $-\pi/2 < \text{ATN}(x) < \pi/2$.

■COS

Cosine

◆Format

COS(x)

Item	Content	Note
x	Angle	Units: Radians

◆Explanation

- Evaluates x and returns its cosine.

■DEG

Get angle in degrees

◆FormatDEG(x)

Item	Content	Note
x	Angle	Units: Radians

◆Explanation

- Converts an angle in radians into degrees and returns the value.

•Example

DEG(PI) Returns the value 180.

■EXPExponentiate with natural base e **◆Format**EXP(x)

Item	Content	Note
x	Exponent	

◆Explanation

- Returns the exponentiation of e (base of natural logarithm) to the power x .

■FP

Get decimal part

◆FormatFP(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates numeric expression x and returns the decimal part of the result.

■INT

Round down to nearest integer

◆Format

INT(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates x , returns the largest integer that does not exceed the result.

• Example

INT(1.3) Returns 1.
INT(-1.3) Returns -2.

■IP

Get integer part

◆Format

IP(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates x and returns the integer part of the result.

• Example

IP(1.3) Returns 1.
IP(-1.3) Returns -1.

■LOG

Calculate natural log

◆Format

LOG(x)

Item	Content	Note
x	Numeric expression	

◆Explanation

- Evaluates x and returns the natural log of the result.
- x must evaluate to a positive value.

■ LOG10

Get common base logarithm

◆ Format

LOG10(x)

Item	Content	Note
x	Numeric expression	

◆ Explanation

- Evaluates x, returns its base 10 logarithm.
- x must evaluate to a positive value.

■ LOG2

Get log base 2

◆ Format

LOG2(x)

Item	Content	Note
x	Numeric expression	

◆ Explanation

- Evaluates x, returns its base 2 logarithm.
- x must evaluate to a positive value.

■ MAX

Returns larger value

◆ Format

MAX(x,y)

Item	Content	Note
x	Numeric expression	
y	Numeric expression	

◆ Explanation

- Evaluates x and y, returns the larger of the two values.

• Example

100 X=MAX(0,X)

If the value of X is greater than 0, it retains its value X; otherwise, it is assigned 0.

110 X=MIN(640,X)

If the value of X is less than 640, it retains its value X; otherwise, it is assigned 640.

■ MIN

Get smaller of two values

◆ Format

MIN(*x*,*y*)

Item	Content	Note
<i>x</i>	Numeric expression	
<i>y</i>	Numeric expression	

◆ Explanation

- Evaluates *x* and *y*, returns the smaller of the two values.

• Example

X=MIN(640,MAX(0,X))

Compare the result of MAX(0,X) with 640 and assign X the smaller of the two.

■ MOD

Get the value of *x* modulo *y*

◆ Format

MOD(*x*,*y*)

Item	Content	Note
<i>x</i>	Numeric expression	
<i>y</i>	Numeric expression	

◆ Explanation

- Evaluate *x* and *y*, return the value of *x* modulo *y*.
- If *y* evaluates to 0, an error occurs.

■ PI

Get value of pi

◆ Format

PI

◆ Explanation

- Returns the value of pi (=3.14159).

■RAD

Returns angle in radians

◆Format

RAD(*x*)

Item	Content	Note
<i>x</i>	Numeric expression	Units: Degrees

◆Explanation

- Converts angle from degrees to radians and returns the value.

■REMAINDER

Return remainder of division

◆Format

REMAINDER(*x*,*y*)

Item	Content	Note
<i>x</i>	Dividend	
<i>y</i>	Divisor	

◆Format

- Divides *x* by *y* and returns the remainder.
- If *y* is zero an error occurs.

■RND

Get random number

◆Format

RND

◆Explanation

- Returns a random number between 0 and 1.

◆Related topics

RANDOMIZE (see p.20)

•Example

INT(RND*100) Returns random integer between 0 and 99.

■SGN

Get sign

◆Format

SGN(*x*)

Item	Content	Note
<i>x</i>	Numeric value	

◆Explanation

- Evaluates *x*, returns its sign.
- If *x* is negative, returns -1, positive returns 1, 0 returns 0.

■SIN

Calculate sine

◆Format

SIN(*x*)

Item	Content	Note
<i>x</i>	Angle	Units: Radians

◆Explanation

- Returns the sine of angle *x*.

■SQR

Get square root

◆Format

SQR(*x*)

Item	Content	Note
<i>x</i>	Numeric expression	

◆Explanation

- Evaluates *X* and returns its square root.
- An error occurs if *X* is a negative number.

TAN

Calculate tangent

◆Format

TAN(*x*)

Item	Content	Note
<i>x</i>	Angle	Units: Radians

◆Explanation

- Returns tangent of angle *x*.

TIME

Get number of seconds since midnight

◆Format

TIME

◆Explanation

- Returns the number of seconds since midnight.

Extended Functions

■ COLLISION

Collision detection between models

◆ Format

COLLISION(*m,s,e*)

Item	Content	Note
<i>m</i>	Model number to collide with	
<i>s</i>	Start model number to check	
<i>e</i>	End model number to check	

◆ Explanation

- For model *m*, a collision check is performed with models *s* to *e*, and the first model number that collides is returned.
- If there is no collision with any model, it returns 0.
- If model number *m* is included within the range of model numbers *s* to *e*, it will be ignored.
- If *s*>*e*, an error occurs.
- Collision detection settings must be configured within the model data.

◆ Related topics

GETCOLL (see p.70)

• Example

N=COLLISION(7,5,10)

Model number 7 is checked for collisions with models 5 to 10, and the number of the model it comes into contact with first is assigned to variable N. No collision check is performed with model number 7 itself.

■ INKEY\$

Get input characters from keyboard

◆ Format

INKEY\$

◆ Explanation

- Returns the character of the currently pressed keyboard key.
- * This is only used when a USB keyboard is connected to the USB port on the "PlayStation2."

◆ Related topics

INPUT (see p.17)

KEYCODE1 (see p.102)

KEYCODE2 (see p.103)

• Example

A\$=INKEY\$

Assigns the currently pressed keyboard character to string A\$.

◆Format

KEYCODE1

◆Explanation

- Returns the key code of currently pressed key.
- If no key is pressed, returns zero.

*This is only used when a USB keyboard is connected to the USB port on the "PlayStation2."

Table 2: Keycode Table

code	Key	code	Key	code	Key	code	Key
0	(No key pressed)	28	Y	56	/	84	/ (Numpad)
1	(Error Code)	29	Z	57	CapsLock	85	* (Numpad)
2		30	1	58	F1	86	- (Numpad)
3		31	2	59	F2	87	+ (Numpad)
4	A	32	3	60	F3	88	Enter (Numpad)
5	B	33	4	61	F4	89	1 (Numpad)
6	C	34	5	62	F5	90	2 (Numpad)
7	D	35	6	63	F6	91	3 (Numpad)
8	E	36	7	64	F7	92	4 (Numpad)
9	F	37	8	65	F8	93	5 (Numpad)
10	G	38	9	66	F9	94	6 (Numpad)
11	H	39	0	67	F10	95	7 (Numpad)
12	I	40	ENTER	68	F11	96	8 (Numpad)
13	J	41	ESC	69	F12	97	9 (Numpad)
14	K	42	Backspace	70	PrintScreen	98	0 (Numpad)
15	L	43	TAB	71	ScrollLock	99	. (Numpad)
16	M	44	Space	72	Break	•	•
17	N	45	—	73	Insert	•	•
18	O	46	^	74	Home	•	•
19	P	47	@	75	PageUp	•	•
20	Q	48	[	76	Delete	•	•
21	R	49		77	End	•	•
22	S	50	]	78	PageDown	•	•
23	T	51	;	79	→	135	↖
24	U	52	:	80	←	136	カタカナひらがな
25	V	53	半角/全角	81	↓	137	¥
26	W	54	,	82	↑	138	Conversion
27	X	55	.	83	NumLock	139	No Conversion

◆Related topics

KEYCODE2 (see p.103)

INKEY\$ (see p.101)

INPUT (see p.17)

•Example

```
100 DO
110 PRINT KEYCODE1
120 LOOP
```

Get key code for currently pressed key and print it.

■KEYCODE2

Get key code for pressed modifier keys

◆Format

KEYCODE2

◆Explanation

- Returns the key code of the currently pressed special key, such as the Shift key or Ctrl key.
 - If nothing is pressed, it returns 0.
 - Returns the sum of the key codes of the special keys pressed simultaneously.
- * This is only effective when a USB keyboard is connected to the USB port on the "PlayStation2".

◆Related topics

INKEY\$ (see p.101) INPUT (see p.17) KEYCODE1 (see p.102)

•Example

```
100 DO
110 PRINT KEYCODE2           Displays the total code value of the special keys being pressed.
120 LOOP
130 END
```

■OBJPOLY

Get number of polygons in model

◆Format

OBJPOLY(x,y)

Item	Content	Note
x	Model buffer number	
y	Object number	

◆Explanation

- Returns the polygon count of object y in model buffer x.

•Example

P=OBJPOLY(1,2) Assigns the number of polygons of object 2 in model buffer 1 to variable P.

◆Format

PAD(x,y)

Item	Content	Note
x	Controller port number	
y	Button number	

◆Explanation

- Gets button press values from a controller.
- Obtains the input information of the controller connected to the controller port specified by x.

Analog Controller(DUALSHOCK 2)

Button: Button number

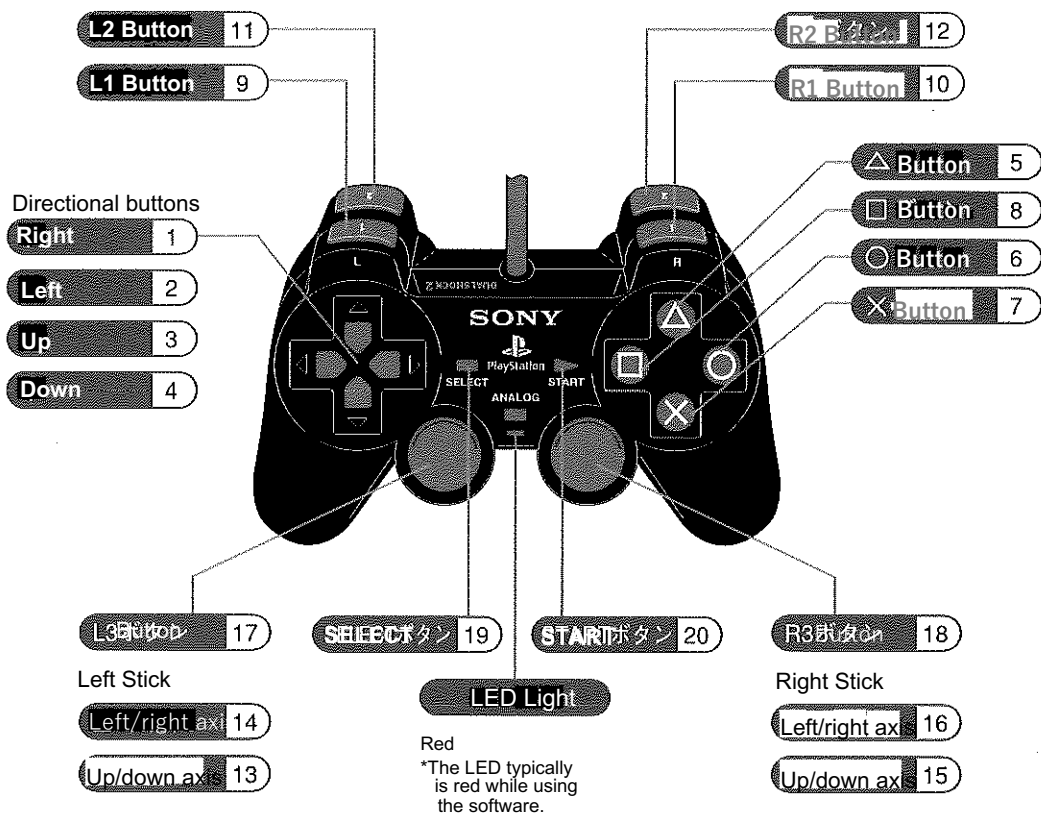


Fig. 19: Controller button numbers

Table 3: Controller button output

Button no.	Button	Range	Idle state	Button no.	Button	Range	Idle state
1	D. pad right	0-255	0	11	L2 Button	0-255	0
2	D. pad left	0-255	0	12	R2 Button	0-255	0
3	D. pad up	0-255	0	13	L Stick (Up/down)	0-255	~127
4	D. pad down	0-255	0	14	L Stick (Left/right)	0-255	~127
5	△ Button	0-255	0	15	R Stick (Up/down)	0-255	~127
6	○ Button	0-255	0	16	R Stick (Left/right)	0-255	~127
7	X Button	0-255	0	17	L3 Button	0-1	0
8	□ Button	0-255	0	18	R3 Button	0-1	0
9	L1 Button	0-255	0	19	SELECT Button	0-1	0
10	R1 Button	0-255	0	20	START Button	0-1	0

•Example

```

100 TEXTMODE 2
110 FOR A=1 TO 16
120   TILEDISP A
130 NEXT A
140 DO
150   CLEAR
160   PRINT " 右 左 上 下 △ ○ X □ L1 R1 L2 R2 L3Y,X R3Y,X"
170   FOR B=1 TO 16
180     D=PAD(1,B)
190     TILECOL B, D,D,B*15, 128
200     TILEPOS B,B*36+5,20,B*36+28,20+D/2, 0
210 NEXT B
220 FSYNC 1
230 LOOP
240 END

```

Check input for buttons 1 to 16
Assign input to variable D
Set color based on input value (pressure).
Set tile primitive position based on input value.

■SCRCHR\$

Get character on screen

◆Format

SCRCHR\$(x, y)

Item	Content	Note
x	Text x coordinate	Range: 0~37(Textmode 1) 0~49(Textmode 2)
y	Text y coordinate	Range: 0~22

◆Explanation

- Gets the character displayed at the position specified by the text coordinates (x,y).
- Specifying a value outside the range of text coordinates will result in an error.

If there is a full-width character in the specified position, the value obtained will be invalid.

•Example

SCRCHR\$(34,12) Gets the character at text coordinates (34,12).

Index

■ Terms

Operator.....	4
Numeric Value.....	5
String.....	5
Array.....	6
Sub-program.....	7
Graphics Specification	32
Color Specification.....	32
About the Screen.....	43
Z-buffer Method.....	44
Z-buffer Issue.....	44
Z-Sort.....	45
About 3D Space Display.....	61
Models.....	62
Camera.....	63
Light Sources.....	64
Fog.....	64

■ Symbols

.....	4
.....	4
*	4
/	4
^.....	4
e scientific notation	4
=.....	4
>.....	4
<.....	4
>=,=>.....	4
<=,=<.....	4
<>,><.....	4
AND.....	4
OR.....	4
NOT	4

■ Commands

ABS	92
ACOS.....	92
ALTEST.....	65
AMBCOL.....	65
ANGLE.....	92
ASIN	93
ATN.....	93
BGCOL.....	46
BGMLOAD.....	82
BGMPLAY.....	82
BGMSTOP.....	83

BSCOL.....	46
CALL.....	9
CAMLENS.....	66
CAMPOS.....	66
CAMROT.....	67
CAMVEC.....	68
CHR\$.....	86
CLEAR	33
COLLISION.....	101
CONTINUE.....	25
COS.....	93
DATE\$	87
DEG	94
DELETE.....	25
DEVICE	33
DIM.....	11
DO-LOOP.....	12
EFFECT.....	47
END.....	13
EXP	94
FILES.....	26
FKEY.....	26
FOGCOL.....	69
FOGSET.....	69
FOR TO STEP.....	14
FP.....	94
FSYNC.....	34
GETCOLL.....	70
GETLPOS.....	71
GETSPOS.....	72
GETWPOS.....	73
GOTO.....	15
IF THEN-END IF.....	16
INIT.....	34
INKEY\$.....	101
INPUT.....	17
INT.....	95
IP.....	95
KEYCODE1.....	102
KEYCODE2.....	103
KILL.....	26
LABEL.....	35
LCASE\$	87
LEN	87
LET.....	18
LIGCOL.....	74
LIGVEC.....	75

LINECOL.....	47
LINEDISP.....	48
LINEOFF.....	48
LINEPOS.....	49
LIST.....	27
LOAD.....	28
LOCATE.....	35
LOG.....	95
LOG 10.....	96
LOG2.....	96
LTRIM\$.....	88
MAX.....	96
MEMORY.....	28
MIN.....	97
MOD.....	97
MODDISP.....	76
MODJOIN.....	77
MODLOAD.....	78
MODOFF.....	78
MODOPT.....	79
MODPOS.....	79
MODROT.....	80
MODSET.....	81
MOUAREA.....	36
MOUCOL.....	36
MOUDISP.....	36
MOUGET.....	37
MOUOFF.....	37
MOUPAT.....	37
MOUSET.....	38
NEW.....	28
OBJPOLY.....	103
ON-GOTO.....	19
OPTION BASE.....	19
ORD.....	88
ORDER.....	38
PAD.....	104
PALCOPY.....	49
PALLOAD.....	50
PI.....	97
POS.....	89
PRINT.....	20
QUIT.....	28
RAD.....	98
RANDOMIZE.....	20
READ/DATA.....	21
REM.....	22
REMAINDER.....	98
RENUMBER.....	29
REPEATS.....	89
RESTORE.....	22

RND.....	98
RTRIM\$.....	90
RUN.....	30
SAVE.....	30
SCRCHR\$.....	105
SCREEN.....	39
SELECT CASE.....	23
SELOAD.....	83
SEPLAY.....	84
SGN.....	99
SIN.....	99
SPRDISP.....	50
SPROFF.....	50
SPRPAL.....	51
SPRPOS.....	52
SPRP0S2.....	53
SPRTEX.....	54
SQR.....	99
STOP.....	24
STR\$.....	90
SUB-END SUB.....	10
TAN.....	100
TEXCOPY.....	55
TEXLOAD.....	55
TEXTCOL.....	40
TEXTMODE.....	40
TILECOL.....	56
TILEDISP.....	56
TILEOFF.....	57
TILEPOS.....	57
TIME.....	100
TIMES.....	90
TRICOL.....	58
TRICOL3.....	59
TRIDISP.....	59
TRIOFF.....	60
TRIPOS.....	60
UCASE\$.....	91
VAL.....	91
VARLOAD.....	41
VARSARE.....	41
VIBRATE.....	42
VIEW.....	31
VOLUME.....	85
ZSORT.....	81



リファレンスマニュアル

株式会社アートディンク

〒261-8701 千葉県美浜区中瀬2-6 WBGマリブウエスト22階

URL <http://www.artdink.co.jp/>

©2001 ARTDINK. All Rights Reserved.

"PlayStation"および"DUALSHOCK"は株式会社ソニー・コンピュータエンタテインメントの登録商標です。

ソニー製"プレイステーション 2"対応プリンター **popegg** 対応。

「popegg」はソニー株式会社の商標です。

その他表記されている商品名、社名は、一般に各社の商標または登録商標です。

2001年4月5日 初版発行

